

INFORMATIQUE • TLE TI

MODULE 1 : PROGRAMMATION

## LEÇON N05

# PROGRAMMATION D'APPLICATIONS MONOPOSTES EN LANGAGE C

DURÉE CONSEILLÉE : 25 H

### DANS CETTE LEÇON

Cette leçon fait passer l'élève de l'algorithmique au véritable développement : écriture, compilation et exécution de programmes en langage C à l'aide d'un IDE. Types, opérateurs, structures de contrôle, fonctions, tableaux, enregistrements et algorithmes classiques (recherche, tri, récursivité) y sont mis en œuvre pour produire des applications monopostes complètes.

- À la fin de cette leçon, je sais déclarer des variables, des constantes et des enregistrements (struct) en C en choisissant les types de base adaptés
- À la fin de cette leçon, je sais lire et afficher des données formatées avec scanf et printf en utilisant stdio.h et les fonctions de math.h
- À la fin de cette leçon, je sais utiliser les opérateurs arithmétiques, relationnels, logiques, d'affectation composée et d'incrémentement en respectant leur priorité
- À la fin de cette leçon, je sais écrire des programmes C utilisant les structures alternatives (if, if...else, switch) et les boucles (for, while, do...while)
- À la fin de cette leçon, je sais déclarer et appeler des fonctions en distinguant variables locales et globales et en maîtrisant la transmission de paramètres
- À la fin de cette leçon, je sais programmer la recherche séquentielle et le tri par sélection sur un tableau en C



### TÉLÉCHARGE ONBUCH

Gratuit • Google Play  
Tuteur IA, annales,  
concours, résultats



### PASSE À ONBUCH+

Tous les cours signés, Tuteur IA  
illimité, fiches PDF — onglet  
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

### COURS

complet, illustré, pas à pas

### MÉTHODES

et exercices résolus

### EXERCICES

type examen + corrigés

### FICHE BILAN

l'essentiel à retenir

**Signé • OnBuch+**

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

**OnBuch+**

## SOMMAIRE

1. Premiers programmes en C : structure, types, entrées-sorties
2. Opérateurs, conversions de types et expressions
3. Structures alternatives et boucles
4. Fonctions, portée des variables et enregistrements
5. Algorithmes classiques : recherche séquentielle et tri par sélection
6. Récursivité, gestion des erreurs et projet d'application complète
7. Activité d'intégration
8. Méthodes et exercices résolus
9. Exercices
10. Corrigés des exercices
11. Fiche bilan

## OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais déclarer des variables, des constantes et des enregistrements (struct) en C en choisissant les types de base adaptés
- À la fin de cette leçon, je sais lire et afficher des données formatées avec scanf et printf en utilisant stdio.h et les fonctions de math.h
- À la fin de cette leçon, je sais utiliser les opérateurs arithmétiques, relationnels, logiques, d'affectation composée et d'incrémentement en respectant leur priorité
- À la fin de cette leçon, je sais écrire des programmes C utilisant les structures alternatives (if, if...else, switch) et les boucles (for, while, do...while)
- À la fin de cette leçon, je sais déclarer et appeler des fonctions en distinguant variables locales et globales et en maîtrisant la transmission de paramètres
- À la fin de cette leçon, je sais programmer la recherche séquentielle et le tri par sélection sur un tableau en C
- À la fin de cette leçon, je sais écrire et tracer une fonction récursive en C
- À la fin de cette leçon, je sais exécuter un programme pas à pas dans un IDE (Code ::Blocks, Dev-C++) et corriger les erreurs de compilation
- À la fin de cette leçon, je sais produire un programme exécutable complet tel qu'une calculatrice ou un gestionnaire de tontine

## PRÉREQUIS

- Notions d'algorithme, de variable et de structure algorithmique vues en Première TI
- Traduction simple d'un algorithme et notion de langage de programmation (début de Terminale)
- Types de données élémentaires (entier, réel, caractère, booléen) et expressions logiques
- Notion de tableau à une dimension et parcours séquentiel
- Fonctions mathématiques usuelles (racine carrée, valeur absolue, cosinus)

## 1 PREMIERS PROGRAMMES EN C : STRUCTURE, TYPES, ENTRÉES-SORTIES

**Situation d'accroche.** Dans son quartier de Yaoundé, Mme Etoundi dirige une coopérative de tontine. Chaque fin de mois, c'est le même rituel : un gros cahier, une calculatrice, et des heures à vérifier les cotisations. Erreurs de calcul dans les totaux, recherches interminables pour retrouver un membre dans la liste, classement des cotisants refait à la main... Son fils, élève en Terminale TI, lui propose de tout automatiser avec un seul programme exécutable. Mais comment traduire en langage C les algorithmes qu'il a appris à écrire en Première ? Comment lire les montants saisis, calculer, trier les membres par ordre de cotisation et retrouver un nom en une fraction de seconde ? C'est toute la puissance de la programmation monoposte en C que cette leçon va dévoiler, jusqu'à la production d'un vrai programme compilé.

**Problématique.** Comment passer d'un algorithme écrit en pseudo-code à un programme C compilé, capable de lire des données au clavier, de calculer et d'afficher des résultats ?

### 1.1 Le langage C : un langage compilé de bas niveau

#### DÉFINITION Programme

Un **programme** est une suite d'instructions écrites dans un langage de programmation, destinée à être exécutée par un ordinateur pour réaliser automatiquement une tâche (calculs, saisies, tris, recherches...). Un programme **monoposte** s'exécute sur une seule machine, sans réseau.

En Première, tu as écrit des algorithmes en pseudo-code : l'ordinateur ne peut pas les exécuter directement. Il faut les traduire dans un langage réel. Le **langage C** est un langage **compilé** et de **bas niveau** : il est proche de la machine (gestion fine de la mémoire, rapidité d'exécution), ce qui le rend idéal pour écrire des programmes performants.

#### DÉFINITION Compilation

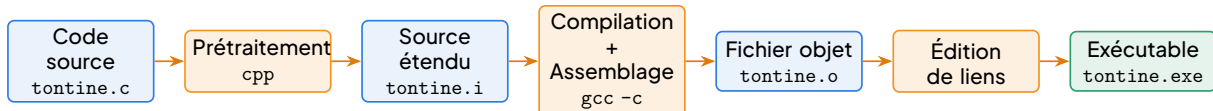
La **compilation** est la traduction complète du code source (fichier `.c`) en langage machine par un programme appelé **compilateur** (par exemple `gcc`). Contrairement à un langage interprété, le programme C est traduit une fois pour toutes : on obtient un fichier exécutable (`.exe` sous Windows) qui peut être lancé sans le code source.

La production d'un exécutable se déroule en quatre étapes principales :

1. **Prétraitement (précompilation)** : le préprocesseur traite les directives commençant par `#` (`#include`, `#define`) et produit un code source étendu (sans commentaires, avec les entêtes insérés et les macros remplacées).
2. **Compilation (proprement dite)** : le compilateur traduit le code source étendu en **code as-**

**sembleur**, puis l'assembleur génère le **fichier objet** (`.o` ou `.obj`) : du code machine non encore lié. Les erreurs de syntaxe sont détectées ici.

3. **Édition de liens** : l'**éditeur de liens** (linker) assemble les fichiers objets et les bibliothèques standards (comme `stdio`, `math`) pour produire l'exécutable final. Les erreurs de référence non résolue (fonction utilisée mais non définie) apparaissent ici.
4. **Exécution** : le système d'exploitation charge l'exécutable en mémoire et lance le programme.



Chaîne de production d'un programme C : du code source à l'exécutable (4 étapes).

#### LESAVAIS-TU ? Le langage C, une légende vivante

Le langage C a été créé en 1972 par Dennis Ritchie aux laboratoires Bell (États-Unis) pour réécrire le système d'exploitation UNIX. Cinquante ans plus tard, il est toujours au cœur de l'informatique mondiale : le noyau Linux est écrit en C, et la plupart des téléphones Android reposent sur ce noyau. Quand un élève de Yaoundé envoie un message WhatsApp, du code C tourne quelque part dans la chaîne !

## 1.2 Prise en main d'un IDE : Code ::Blocks

Pour écrire, compiler et exécuter un programme C, on utilise un **IDE** (Environnement de Développement Intégré) comme **Code ::Blocks** ou **Dev-C++**, qui regroupe l'éditeur de texte, le compilateur (généralement GCC via MinGW sous Windows) et le débogueur.

#### MÉTHODE Créer et exécuter un premier programme sous Code ::Blocks

1. **Créer un projet** : menu `File > New > Project`, choisir `Console application`, langage **C**, puis donner un nom au projet (ex. `Tontine`). Choisir le compilateur `GNU GCC Compiler`.
2. **Écrire le code** : ouvrir le fichier `main.c` dans l'éditeur et saisir le programme.
3. **Compiler** : appuyer sur `F9` (`Build`) ; la fenêtre `Build log` affiche les erreurs éventuelles avec le numéro de ligne. Si le projet utilise `math.h`, vérifier que l'option `Linker settings > Add contient m (libm)` — souvent automatique.
4. **Exécuter** : appuyer sur `Ctrl+F10` (ou `F9` si tout est déjà compilé) ; une console noire s'ouvre et affiche les résultats.
5. **Exécuter pas à pas** : placer un point d'arrêt (clic à gauche du numéro de ligne), lancer

le débogueur avec F8, puis avancer instruction par instruction avec F7 en observant les valeurs des variables dans la fenêtre Watches.

L'exécution pas à pas avec le débogueur est un savoir-faire essentiel : elle permet de voir le programme vivre, de comprendre l'ordre d'exécution des instructions et de traquer les bugs.

### 1.3 Structure minimale d'un programme C

Tout programme C, même le plus petit, respecte une architecture fixe :

```
#include <stdio.h> // bibliotheque des entrees-sorties

int main()          // fonction principale : point d'entree du programme
{
    printf("Bonjour la Tontine !\\n"); // instruction d'affichage
    return 0;        // le programme signale qu'il s'est bien termine
}
```

#### DÉFINITION Bibliothèque

Une **bibliothèque** est un ensemble de fonctions prêtes à l'emploi, regroupées dans un fichier d'en-tête (.h). La directive `#include <stdio.h>` donne accès aux fonctions d'entrée-sortie `printf` et `scanf`; `#include <math.h>` donne accès aux fonctions mathématiques (`sqrt`, `pow`, ...).

#### PROPRIÉTÉ Règles de structure d'un programme C

- La fonction `main()` est le point d'entrée : l'exécution commence toujours par sa première instruction.
- Le corps d'une fonction est délimité par des accolades `{ ... }`.
- Chaque instruction se termine par un point-virgule ;.
- `return 0;` indique au système que le programme s'est terminé sans erreur (code de retour `EXIT_SUCCESS`).
- Le C est sensible à la casse : `Main` et `main` sont différents.

#### ATTENTION Les deux erreurs du débutant

- Oublier le point-virgule en fin d'instruction : le compilateur signale une erreur syntax error souvent à la ligne suivante. Vérifie toujours la ligne indiquée et la précédente.

- Oublier l'esperluette & dans `scanf` : le programme compile parfois sans erreur mais plante à l'exécution (erreur de segmentation SIGSEGV) ou lit des valeurs absurdes. Détail plus bas.

## 1.4 Types de base, variables et constantes

Un programme manipule des données : montants en FCFA, âges, notes, noms... Chaque donnée doit être rangée dans une **variable** d'un **type** adapté, car le type détermine la place occupée en mémoire et les opérations autorisées.

| Type   | Taille usuelle | Plage / usage                             | Exemple           |
|--------|----------------|-------------------------------------------|-------------------|
| int    | 4 octets       | entiers de -2 147 483 648 à 2 147 483 647 | nombre de membres |
| float  | 4 octets       | réels, environ 7 chiffres significatifs   | une note sur 20   |
| double | 8 octets       | réels, environ 15 chiffres significatifs  | montant en FCFA   |
| char   | 1 octet        | un caractère (code ASCII)                 | 'M' ou 'F'        |

**Choisir le bon type.** Pour l'âge d'un membre, un `int` suffit. Pour un montant de cotisation comme 25 000,50 FCFA, on préfère `double` (plus précis que `float`). Pour une note sur 20, `float` convient. Pour le sexe d'un membre, un `char`.

### DÉFINITION Identificateur

Un **identificateur** est le nom donné à une variable, une constante ou une fonction. Règles en C : il commence par une lettre ou un souligné `_`, ne contient que des lettres, chiffres et `_`, et ne doit pas être un mot réservé (`int`, `return`, `if`...). Convention : `montantCotisation`, `age_membre`.

### PROPRIÉTÉ Déclaration de variables et de constantes

- **Variable** : `type nom;` ou `type nom = valeur;` (déclaration avec initialisation).
- **Constante préprocesseur** : `#define TVA 0.1925` (le compilateur remplace chaque `TVA` par sa valeur ; pas de point-virgule!).
- **Constante typée** : `const double TVA = 0.1925;` (recommandé : le type est vérifié, portée respectée).

```
#include <stdio.h>
#define TAUX_TVA 0.1925 // constante preprocesseur

int main()
{
```

```
const int NB_MEMBRES_MAX = 50; // constante typee
int age = 17; // entier
double cotisation = 25000.50; // reel double precision
char sexe = 'M'; // caractere (apostrophes simples)
return 0;
}
```

## 1.5 Sorties formatées avec printf

La fonction `printf` affiche du texte et des valeurs à l'écran. Les **spécificateurs de format** indiquent où et comment insérer les valeurs :

| Spécificateur | Effet                                                |
|---------------|------------------------------------------------------|
| %d            | entier (int)                                         |
| %f            | réel (float ou double), 6 décimales par défaut       |
| %.2f          | réel arrondi à 2 décimales (idéal pour les montants) |
| %c            | caractère (char)                                     |
| %s            | chaîne de caractères                                 |
| \n            | saut de ligne; \t : tabulation                       |

```
double total = 15230.4567;
printf("Total : %.2f FCFA\n", total); // affiche : Total : 15230.46 FCFA
printf("Nom\tCotisation\n"); // colonnes alignees par tabulation
```

### ATTENTION

Utiliser %d pour afficher un double (ou l'inverse) produit des valeurs aberrantes sans message d'erreur. Le spécificateur doit toujours correspondre au type de la variable.

## 1.6 Entrées formatées avec scanf

La fonction `scanf` lit ce que l'utilisateur tape au clavier et le range dans une variable. Point capital : `scanf` a besoin de l'**adresse** de la variable, d'où l'**esperluette &** devant son nom.

```
int age;
double montant;
printf("Entrez votre age : ");
scanf("%d", &age); // & devant la variable : OBLIGATOIRE
printf("Entrez le montant : ");
scanf("%lf", &montant); // %lf pour un double, %f pour un float
```

### ATTENTION

#### Piège classique : %f versus %lf

Avec `printf`, %f convient pour float et double (promotion automatique). Avec `scanf`, il faut

`%f` pour un float mais `%lf` pour un double. Oublier le `l` donne des lectures fausses silencieuses.

**ATTENTION** Buffer d'entrée et retour de `scanf`

- `scanf` laisse le caractère `\n` (Entrée) dans le tampon. Un `scanf("%c", &c)` suivant lira ce `\n` au lieu du caractère attendu. Parade : mettre un espace avant `%c` (" %c") pour ignorer les blancs, ou vider le buffer avec `while(getchar() != '\n');`.
- `scanf` renvoie le nombre d'éléments lus avec succès. Testez-le : `if (scanf("%lf", &x) != 1) { /* erreur saisie */ }`.

**ATTENTION** Affichage du prompt avant saisie

Sur certains systèmes (Linux), la sortie standard est tamponnée par ligne : un `printf("Entrez x : ")` sans `\n` final n'apparaît pas avant le `scanf` qui suit. Bonne pratique : ajouter `fflush(stdout);` après le `printf` de prompt.

C:\Tontine\tontine.exe

```
Entrez le prix HT : 10000
Prix HT : 10000.00 FCFA
TVA 19.25% : 1925.00 FCFA
Prix TTC : 11925.00 FCFA

Process returned 0 (0x0)
Press any key to continue.
```

Fenêtre console stylisée : dialogue `printf/scanf` du programme de calcul du prix TTC.

## 1.7 La bibliothèque `math.h`

Pour les calculs mathématiques, on inclut `#include <math.h>`. Fonctions utiles au programme de Terminale :

| Fonction                    | Rôle                                                                                   |
|-----------------------------|----------------------------------------------------------------------------------------|
| <code>sqrt(x)</code>        | racine carrée de $x$ (avec $x \geq 0$ )                                                |
| <code>fabs(x)</code>        | valeur absolue d'un réel; <code>abs(n)</code> pour un entier ( <code>stdlib.h</code> ) |
| <code>pow(x, y)</code>      | $x$ élevé à la puissance $y$                                                           |
| <code>cos(x), sin(x)</code> | cosinus et sinus, $x$ en radians                                                       |

#### EXEMPLE Calcul du discriminant d'un trinôme

Pour résoudre  $ax^2 + bx + c = 0$ , on calcule  $\Delta = b^2 - 4ac$  :

```
#include <stdio.h>
#include <math.h>

int main()
{
    double a = 1, b = -5, c = 6;
    double delta = pow(b, 2) - 4 * a * c; // ou b*b - 4*a*c
    printf("Delta = %.2f\\", delta);      // Delta = 1.00
    if (delta >= 0) {
        double x1 = (-b - sqrt(delta)) / (2 * a);
        double x2 = (-b + sqrt(delta)) / (2 * a);
        printf("x1 = %.2f x2 = %.2f\\", x1, x2);
    }
    return 0;
}
```

## 1.8 Exemple chiffré : du prix HT au prix TTC

#### EXEMPLE Programme de calcul du prix TTC (TVA 19,25 %)

Au Cameroun, la TVA est de 19,25 %. Ce programme saisit un prix hors taxe et affiche le prix toutes taxes comprises, arrondi au centime grâce à `%.2f` :

```
#include <stdio.h>
#define TVA 0.1925 // taux de TVA au Cameroun

int main()
{
    double prixHT, montantTVA, prixTTC;

    printf("Entrez le prix HT : ");
    scanf("%lf", &prixHT); // lecture d'un double

    montantTVA = prixHT * TVA;
    prixTTC = prixHT + montantTVA;

    printf("Prix HT : %.2f FCFA\\", prixHT);
    printf("TVA : %.2f FCFA\\", montantTVA);
    printf("Prix TTC : %.2f FCFA\\", prixTTC);
}
```

```
    return 0;  
}
```

Pour un prix HT de 10 000 FCFA :  $TVA = 10\,000 \times 0,1925 = 1\,925$  FCFA, donc prix TTC = 11 925,00 FCFA.

## 1.9 Exercice résolu et TP guidé

### EXERCICE RÉSOLU 1 La moyenne de deux notes

Énoncé : écrire un programme C complet qui saisit deux notes (réelles) d'un élève de Terminale TI, calcule leur moyenne et l'affiche avec deux décimales.

#### SOLUTION

Solution détaillée : on déclare trois variables `float` (une note sur 20 n'exige pas la précision du `double`), on lit les deux notes avec `scanf` et `&`, on calcule la moyenne en veillant à écrire `2.0` (division réelle) et non `2` :

```
#include <stdio.h>  
  
int main()  
{  
    float note1, note2, moyenne;  
  
    printf("Entrez la premiere note : ");  
    scanf("%f", &note1);  
    printf("Entrez la deuxieme note : ");  
    scanf("%f", &note2);  
  
    moyenne = (note1 + note2) / 2.0; // 2.0 : division reelle  
  
    printf("La moyenne est : %.2f / 20\\n", moyenne);  
    return 0;  
}
```

Trace d'exécution : pour `note1 = 12` et `note2 = 15`, on obtient La moyenne est : 13.50 / 20. Piège évité : écrire `/ 2` avec des `float` fonctionne ici, mais avec deux `int`, `(a + b) / 2` serait une division entière tronquée — nous y reviendrons dans la section sur les conversions de types.

### EXPÉRIENCE TP : ton premier programme compilé

Objectif : créer, compiler et exécuter pas à pas le programme de la moyenne. Matériel : un poste avec Code ::Blocks installé.

1. Crée un projet `Moyenne` (Console application, langage C).
2. Recopie le code de l'exercice résolu dans `main.c`.
3. Compile avec F9 : corrige toute erreur signalée dans le Build log (vérifie les points-virgules, les & dans `scanf`).
4. Exécute avec Ctrl+F10 et teste avec les notes 12 et 15.
5. Place un point d'arrêt sur la ligne du premier `scanf`, lance le débogueur (F8) et avance avec F7 : observe dans les Watches les variables `note1`, `note2` puis `moyenne` se remplir.

Observation : avant le `scanf`, les variables contiennent des valeurs aléatoires (mémoire non initialisée) ; après, elles contiennent exactement ce qui a été tapé. Interprétation : une variable non initialisée n'est jamais nulle par magie — d'où l'importance d'initialiser ou de lire toute variable avant de l'utiliser.

#### À RETENIR L'essentiel de la section

- Le C est un langage compilé : source `.c` → prétraitement → compilation/assemblage → objet `.o` → édition de liens → exécutable.
- Tout programme C contient `#include <stdio.h>` et une fonction `main()` dont le corps est entre accolades ; chaque instruction finit par `;`.
- Types de base : `int`, `float`, `double`, `char` ; constantes avec `#define` ou `const`.
- `printf` affiche avec `%d, %.2f, %c, %s` ; `scanf` lit avec l'esperluette & (et `%lf` pour un `double`).
- `math.h` fournit `sqrt`, `pow`, `fabs`, `cos`, `sin`.
- Attention au buffer clavier (`\n` résiduel) et au tampon de sortie (`fflush(stdout)`).

#### EXERCICE 1 À toi de jouer

Écris un programme C qui saisit le montant mensuel de cotisation d'un membre de la ton-tine de Mme Etoundi (en FCFA, type `double`) et le nombre de mois (type `int`), puis affiche le total cotisé sur l'année, formaté avec deux décimales. Teste-le avec 5 000 FCFA sur 12 mois : tu dois obtenir 60000.00 FCFA.

## 2 OPÉRATEURS, CONVERSIONS DE TYPES ET EXPRESSIONS

Dans la section précédente, tu as appris à écrire tes premiers programmes en C : déclarer des variables de types de base (`int`, `float`, `double`, `char`), lire des données avec `scanf` et afficher des résultats avec `printf`. Mais un programme qui se contente de stocker et d'afficher des valeurs ne fait pas grand-chose d'utile. La véritable puissance d'un programme, c'est de calculer, de comparer et de prendre des décisions. Imagine la caisse d'une boutique à Douala : le logiciel doit additionner les prix, appliquer une remise de 5 % si le total dépasse 50000 FCFA, vérifier que le client est majeur pour certains produits... Toutes ces opérations reposent sur les **opérateurs** et les **expressions**. C'est l'objet de cette section.

### 2.1 Opérandes, expressions et opérateurs arithmétiques

#### DÉFINITION Opérande, opérateur, expression

Une **opérande** est une valeur sur laquelle porte un calcul : une constante (5), une variable (`prix`) ou le résultat d'un sous-calcul. Un **opérateur** est un symbole qui combine une ou deux opérandes (+, \*, >, ...). Une **expression** est un assemblage d'opérateurs et d'opérandes qui possède une valeur après évaluation : `3 * note + 2` est une expression dont la valeur dépend de `note`.

Le langage C fournit les cinq opérateurs arithmétiques classiques :

| Opérateur | Nom            | Exemple            | Résultat    |
|-----------|----------------|--------------------|-------------|
| +         | Addition       | <code>7 + 2</code> | 9           |
| -         | Soustraction   | <code>7 - 2</code> | 5           |
| *         | Multiplication | <code>7 * 2</code> | 14          |
| /         | Division       | <code>7 / 2</code> | 3 (entier!) |
| %         | Modulo (reste) | <code>7 % 2</code> | 1           |

#### ATTENTION Division entière contre division réelle

En C, le résultat de / dépend du type des opérandes, pas de celui de la variable qui reçoit le résultat. Si les deux opérandes sont entières, la division est **entière** (troncature) : `7 / 2` vaut 3, tandis que `7 / 2.0` vaut 3,5. Piège classique du Baccalauréat :

```
float moyenne;  
moyenne = 15 / 2;    /* moyenne vaut 7.0 et non 7.5 ! */  
moyenne = 15 / 2.0; /* moyenne vaut 7.5 : division réelle */  
moyenne = 15.0 / 2; /* idem : 7.5 */
```

De même, l'opérateur % n'existe que pour les entiers : écrire `7.5 % 2` provoque une erreur

de compilation.

#### LESAVAIS-TU ? Le modulo, un outil du quotidien

L'opérateur % est partout. Pour tester la parité d'un nombre, on écrit `n % 2 == 0`. Pour gérer des cycles, c'est irremplaçable : si aujourd'hui est le jour numéro  $j$  de la semaine (0 = lundi), alors dans  $n$  jours ce sera le jour  $(j + n) \% 7$ . Les systèmes de Mobile Money utilisent exactement ce principe pour planifier des rappels périodiques, et ton emploi du temps hebdomadaire au lycée fonctionne pareil !

## 2.2 Conversions de types : implicite et explicite (cast)

Que se passe-t-il quand une expression mélange des types différents, par exemple un `int` et un `float` ? Le C ne refuse pas : il convertit.

#### DÉFINITION Conversion implicite et conversion explicite

La **conversion implicite** (ou promotion) est réalisée automatiquement par le compilateur : dans une expression mixte, l'opérande de type « plus petit » est convertie vers le type « plus grand » selon la hiérarchie `char` → `int` → `float` → `double`. Aucune information n'est perdue dans ce sens. La **conversion explicite** (ou cast) est demandée par le programmeur avec la syntaxe :

(type\_vise) expression

Elle peut provoquer une perte d'information : `(int) 3.99` vaut 3 (troncature, pas arrondi).

#### EXEMPLE Forcer une division réelle grâce au cast

Cas typique : calculer la moyenne de deux quantités stockées dans des entiers.

```
#include <stdio.h>

int main(void)
{
    int somme = 27, effectif = 4;
    float m1, m2, m3;

    m1 = somme / effectif;           /* 6.0 : division entiere ! */
    m2 = (float)somme / effectif;   /* 6.75 : somme devient 27.0 */
    m3 = (float)(somme / effectif); /* 6.0 : cast trop tard ! */

    printf("m1 = %.2f, m2 = %.2f, m3 = %.2f\n", m1, m2, m3);
    return 0;
}
```

```
| }
```

Pour `m3`, le cast s'applique après la division entière : le mal est déjà fait. Il faut convertir au moins une opérande avant la division.

## 2.3 Opérateurs relationnels et logiques

Pour prendre des décisions (section suivante), il faut pouvoir comparer des valeurs. Les opérateurs relationnels sont `==` (égal), `!=` (différent), `<`, `<=`, `>`, `>=`. Une comparaison produit un `int` valant 1 (vrai) ou 0 (faux) : historiquement, le langage C ne possède pas de type booléen natif (la bibliothèque `stdbool.h`, ajoutée plus tard, fournit `bool`, `true` et `false`, mais elle n'est pas exigée au programme).

### PROPRIÉTÉ Vérité en C (admise, vérifiée sur exemples)

En C, toute valeur nulle (0, 0.0) est considérée comme fausse, et toute valeur non nulle est considérée comme vraie. Ainsi `5 > 3` vaut 1, `5 == 3` vaut 0, et une expression comme `7` seule est « vraie ». On peut donc écrire `if (stock)` au lieu de `if (stock != 0)`.

### ATTENTION Le piège n°1 du débutant : `=` contre `==`

`=` est l'affectation (elle range une valeur), `==` est la comparaison (elle teste). Le code suivant compile sans erreur mais est faux :

```
int age = 15;
if (age = 18) { /* ERREUR : on AFFECTE 18 à age ! */
    /* age vaut désormais 18, et 18 est "vrai" :
       le test réussit TOUJOURS */
}
```

Conséquence de la propriété précédente : l'expression `age = 18` vaut 18, donc vrai. Astuce de professionnel : écrire `if (18 == age)` — si tu oublies un `=`, le compilateur refusera `18 = age`, car on ne peut pas affecter une valeur à une constante.

Les opérateurs logiques combinent des conditions : `&&` (ET), `||` (OU), `!` (NON).

| a | b | a && b | a    b | !a |
|---|---|--------|--------|----|
| 0 | 0 | 0      | 0      | 1  |
| 0 | 1 | 0      | 1      | 1  |
| 1 | 0 | 0      | 1      | 0  |
| 1 | 1 | 1      | 1      | 0  |

**PROPRIÉTÉ** Évaluation paresseuse

Le C évalue les conditions de gauche à droite et s'arrête dès que le résultat est connu : pour `a && b`, si `a` est faux, `b` n'est pas évalué ; pour `a || b`, si `a` est vrai, `b` n'est pas évalué. Cela permet d'écrire en toute sécurité :

```
if (n != 0 && 100 / n > 5) { /* pas de division par zero : */
    /* si n == 0, le second test n'est jamais calculé */
}
```

**2.4 Affectations composées, incrémentation et décrémentation**

Modifier une variable à partir de sa propre valeur est si fréquent que le C offre des raccourcis :

| Affectation composée | Équivalent              |
|----------------------|-------------------------|
| <code>x += 3;</code> | <code>x = x + 3;</code> |
| <code>x -= 3;</code> | <code>x = x - 3;</code> |
| <code>x *= 2;</code> | <code>x = x * 2;</code> |
| <code>x /= 2;</code> | <code>x = x / 2;</code> |
| <code>x %= 7;</code> | <code>x = x % 7;</code> |

Les opérateurs `++` et `--` ajoutent ou retranchent 1. Mais leur position change tout :

**EXEMPLE** Pré-incrémentation contre post-incrémentation

- `++i` (pré) : on incrémente d'abord, puis on utilise la nouvelle valeur ;
- `i++` (post) : on utilise l'ancienne valeur d'abord, puis on incrémente.

```
int i, j;

i = 5;
j = i++ * 2; /* j reçoit 5*2 = 10, PUIS i devient 6 */
/* Resultat : j = 10, i = 6 */

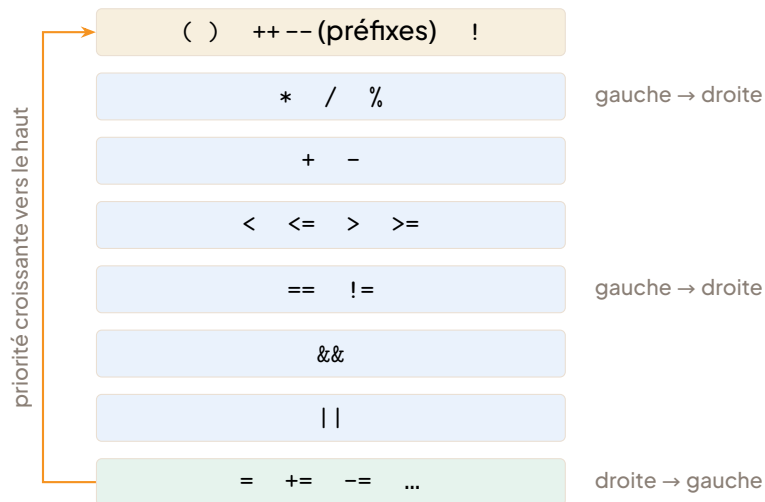
i = 5;
j = ++i * 2; /* i devient 6 D'ABORD, puis j reçoit 6*2 */
/* Resultat : j = 12, i = 6 */
```

Utilisés seuls sur une ligne (`i++;`), les deux formes sont strictement équivalentes.

**2.5 Priorité et associativité des opérateurs**

Que vaut `3 + 4 * 2`? Comme en mathématiques, le C applique des règles de **priorité** : la multiplication d'abord, donc `3 + 8 = 11`. L'**associativité** tranche les égalités de priorité : `20 / 5 / 2`

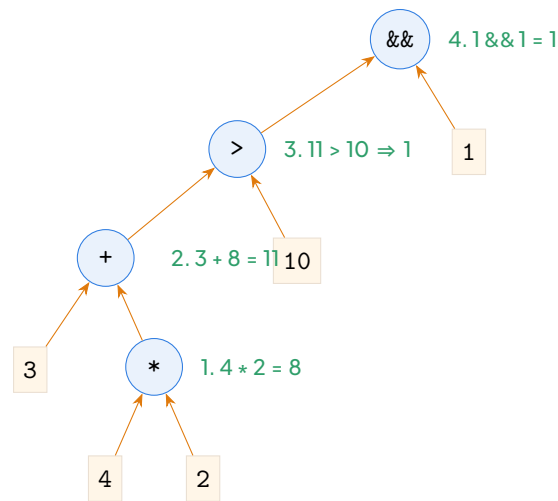
s'évalue de gauche à droite, donc  $(20/5)/2 = 2$ .



Hiérarchie des opérateurs du C, du plus prioritaire (en haut) au moins prioritaire (en bas). Les parenthèses priment sur tout.

#### MÉTHODE Évaluer une expression pas à pas

1. Évaluer d'abord le contenu des parenthèses, de la plus interne à la plus externe ;
  2. appliquer les `++/--` et le `!` ;
  3. effectuer les `*`, `/`, `%` de gauche à droite ;
  4. effectuer les `+`, `-` de gauche à droite ;
  5. effectuer les comparaisons (`<`, `<=`, `>`, `>=`, puis `==`, `!=`) ;
  6. appliquer `&&`, puis `||` ;
  7. effectuer l'affectation en dernier.
- En cas de doute, mets des parenthèses : elles ne coûtent rien et rendent le code lisible.



Arbre d'évaluation de l'expression  $3 + 4 * 2 > 10 \&\& 1$  : les feuilles sont les opérandes, chaque nœud est évalué après ses enfants, dans l'ordre indiqué.

#### EXERCICE RÉSOLU 2 Trace à la main d'une expression complexe

Énoncé : soit `int a = 10, b = 4, r;` et l'instruction `r = a % b + a / b * 2 > 5 && b != 0;`. Donner la valeur finale de `r` en détaillant chaque étape.

#### SOLUTION

Solution détaillée : on applique la méthode des priorités.

1. `%`, `/`, `*` d'abord, de gauche à droite : `a % b = 10 % 4 = 2`; `a / b = 10 / 4 = 2` (division entière); `2 * 2 = 4`.
2. Addition : `2 + 4 = 6`.
3. Comparaisons : `6 > 5` vaut 1; `b != 0` s'écrit `4 != 0`, qui vaut 1.
4. `&&` : `1 && 1` vaut 1.
5. Affectation : `r = 1`.

La valeur finale de `r` est donc 1. Remarque : grâce à l'évaluation paresseuse, si le premier membre du `&&` avait été faux, `b != 0` n'aurait même pas été évalué.

#### EXERCICE 2 À toi de jouer

Soit `int x = 7, y = 3, z;` et `float f;`. Chaque question repart des valeurs initiales `x = 7` et `y = 3`.

1. Donner la valeur de  $z$  après :  $z = x / y + x \% y * 5;$
2. Donner les valeurs de  $x$ ,  $y$  et  $z$  après :  $z = ++x - y++;$
3. Donner la valeur de  $f$  après :  $f = (\text{float})x / y;$
4. L'expression  $x > y \ \&\& \ y > 0 \ || \ x == 0$  vaut-elle 0 ou 1?

**CORRIGÉ** Exercice : trace d'expressions

1.  $x / y = 2$  (division entière),  $x \% y = 1$ , donc  $z = 2 + 1 \times 5 = 7$ .
2.  $++x$  :  $x$  devient 8 puis est utilisé ;  $y++$  : on utilise 3 puis  $y$  devient 4. Donc  $z = 8 - 3 = 5$ ,  $x = 8$ ,  $y = 4$ .
3. Le cast transforme  $x$  en 7.0 avant la division :  $f = 7,0/3 \approx 2,33$ .
4.  $x > y$  vaut 1 et  $y > 0$  vaut 1, donc  $1 \ \&\& \ 1$  vaut 1 ; comme  $\&\&$  est prioritaire sur  $||$ , le résultat est  $1 \ || \ 0 = 1$ .

**À RETENIR** L'essentiel de la section

- $/$  entre entiers tronque ;  $\%$  donne le reste et n'existe que pour les entiers.
- Conversion implicite : vers le type le plus grand ; cast  $(\text{type})\text{expr}$  : à placer avant l'opération critique.
- Toute valeur non nulle est vraie, 0 est faux ; ne jamais confondre  $=$  et  $==$ .
- $\&\&$  et  $||$  s'évaluent paresseusement, de gauche à droite.
- $i++$  utilise puis incrémente ;  $++i$  incrémente puis utilise.
- Priorité : parenthèses, puis  $++/--/!$ , puis  $*$  /  $\%$ , puis  $+$  /  $-$ , puis comparaisons, puis  $\&\&$ , puis  $||$ , puis affectation.

Tu maîtrises désormais l'écriture et l'évaluation des expressions : c'est le carburant de tout programme C. Dans la prochaine section, nous mettrons ces expressions au service de la décision et de la répétition avec les structures alternatives (`if`, `switch`) et les boucles (`for`, `while`, `do...while`).

### 3 STRUCTURES ALTERNATIVES ET BOUCLES

Dans la section précédente, tu as appris à construire des expressions : calculs arithmétiques, comparaisons, conditions logiques. Mais jusqu'ici, nos programmes s'exécutaient toujours de

la même façon : ligne après ligne, du haut vers le bas, sans jamais hésiter ni recommencer. Or un programme utile doit pouvoir prendre des décisions (« si la note est supérieure ou égale à 10, afficher Admis ») et répéter des tâches (« calculer la moyenne des 25 élèves de la classe »). C'est exactement le rôle des **structures alternatives** et des **boucles**, que nous allons étudier maintenant. Ce sont les outils qui transforment une simple suite de calculs en un véritable programme intelligent.

### 3.1 Les structures alternatives : `if` et `if...else`

**Intuition.** Imagine le système de délibération d'un conseil de classe : si la moyenne est au moins 10, l'élève est admis ; sinon, il est ajourné. Le programme doit choisir un chemin parmi deux, selon une condition. C'est ce que réalise l'instruction `if`.

#### DÉFINITION Structure alternative

Une **structure alternative** est une instruction qui permet d'exécuter un bloc d'instructions uniquement si une condition est vraie. En C, une condition est une expression dont la valeur est interprétée comme vraie si elle est différente de 0, et fausse si elle vaut 0.

#### PROPRIÉTÉ Syntaxe du `if` simple et du `if...else`

```
if (condition){
    // instructions exécutées si condition est vraie
}

if (condition){
    // instructions si vrai
} else {
    // instructions si faux
}
```

La condition est toujours entre parenthèses. Si le bloc ne contient qu'une seule instruction, les accolades sont facultatives, mais nous te conseillons vivement de les mettre systématiquement : cela évite de nombreux bugs lors des modifications ultérieures du code.

#### EXEMPLE Décision d'admission au Baccalauréat

```
#include <stdio.h>

int main(void){
    float moyenne;
    printf("Entrez la moyenne du candidat : ");
    scanf("%f", &moyenne);

    if (moyenne >= 10){
```

```
    printf("Candidat ADMIS.\n");
} else {
    printf("Candidat AJOURNE.\n");
}
return 0;
}
```

Ici, la condition `moyenne >= 10` utilise un opérateur relationnel vu à la section précédente. Un seul des deux blocs est exécuté, jamais les deux.

**Conditions composées.** On combine plusieurs tests avec les opérateurs logiques `&&` (ET), `||` (OU) et `!` (NON). Par exemple, la mention « Bien » s’obtient avec une moyenne entre 14 et 16 :

```
if (moyenne >= 14 && moyenne < 16){
    printf("Mention Bien.\n");
}
```

**ATTENTION** Ne jamais écrire `if (a = 10)`

L’erreur classique du débutant est d’écrire `if (moyenne = 10)` avec un seul `=`. Ce n’est pas une comparaison mais une affectation : la variable reçoit 10, et l’expression vaut 10, donc elle est considérée vraie ! Le bloc s’exécute toujours. La comparaison d’égalité s’écrit `==`. De même, ne place jamais de point-virgule juste après la parenthèse du `if` : `if (x > 0);` crée un corps vide et le bloc qui suit s’exécute dans tous les cas.

### 3.2 Imbrication des `if` et problème du `else` pendu

On peut placer un `if` à l’intérieur d’un autre : c’est l’**imbrication**. C’est indispensable pour traiter plusieurs cas, comme le calcul des mentions.

**EXEMPLE** Calcul de la mention avec `if` imbriqués

```
if (moyenne >= 16){
    printf("Mention Tres Bien\n");
} else if (moyenne >= 14){
    printf("Mention Bien\n");
} else if (moyenne >= 12){
    printf("Mention Assez Bien\n");
} else if (moyenne >= 10){
    printf("Mention Passable\n");
} else {
    printf("Ajourné\n");
}
```

Les conditions sont évaluées dans l’ordre : dès qu’une est vraie, son bloc s’exécute et toutes les autres sont ignorées. L’ordre des tests est donc crucial : tester d’abord `moyenne`

`>= 10` rendrait les autres branches inaccessibles.

**ATTENTION** Le `else` pendu

En C, un `else` se rattache toujours au `if` le plus proche qui n'a pas encore de `else`. Dans le code suivant, l'indentation trompe le lecteur :

```
if (note >= 10)
    if (note >= 12)
        printf("Bien\n");
else
    printf("Ajourné\n"); // rattache au 2e if, pas au 1er !
```

Le `else` appartient au second `if`, alors que l'indentation suggère le contraire. Bonne pratique : mets toujours des accolades et indente rigoureusement (décalage constant de 4 espaces) pour que la structure du code reflète sa logique.

### 3.3 La structure `switch` : choix multiples

**Intuition.** Quand une même variable peut prendre plusieurs valeurs précises (1, 2, 3, 4 pour un menu), enchaîner des `if...else if` devient lourd. La structure `switch` est faite pour cela : elle compare la valeur d'une expression à une liste de cas.

**PROPRIÉTÉ** Syntaxe du `switch`

```
switch(expression) { // expression de type entier ou char
    case valeur1 :
        // instructions
        break;
    case valeur2 :
        // instructions
        break;
    default : // facultatif : tous les autres cas
        // instructions
}
```

L'expression doit être de type entier (`int`, `char`) : on ne peut pas faire un `switch` sur un `float` ni sur une chaîne de caractères. Le `default` traite tous les cas non prévus.

**EXEMPLE** Menu de calculatrice

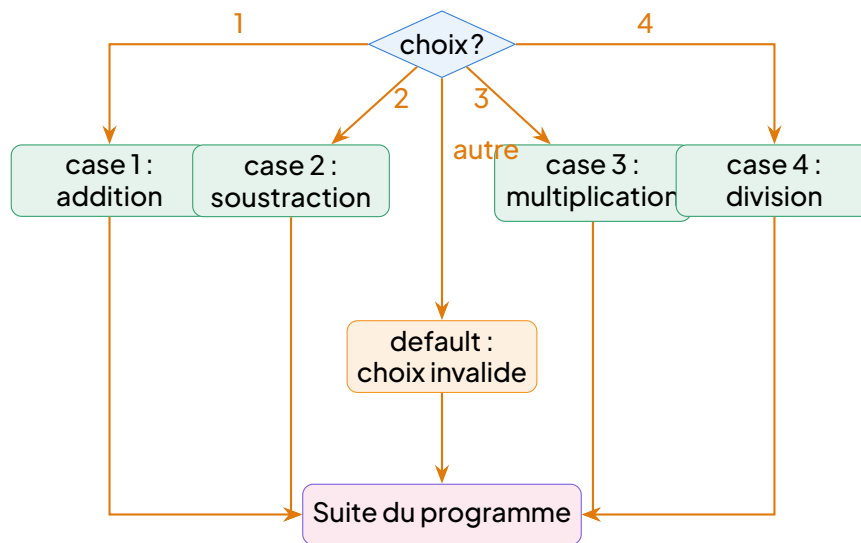
```
int choix;
float a, b;
```

```
printf("1. Addition\n2. Soustraction\n3. Multiplication\n4. Division\n");
printf("Votre choix : ");
scanf("%d", &choix);
printf("Entrez deux nombres : ");
scanf("%f %f", &a, &b);

switch (choix){
    case 1:
        printf("Resultat : %.2f\n", a + b);
        break;
    case 2:
        printf("Resultat : %.2f\n", a - b);
        break;
    case 3:
        printf("Resultat : %.2f\n", a * b);
        break;
    case 4:
        if (b != 0) {
            printf("Resultat : %.2f\n", a / b);
        } else {
            printf("Erreur : division par zero !\n");
        }
        break;
    default:
        printf("Choix invalide.\n");
}
```

**ATTENTION** Oublier le `break` : le fall-through

Si tu oublies le `break` à la fin d'un `case`, l'exécution continue dans le cas suivant sans tester sa condition : c'est le fall-through (passage en cascade). Avec `choix = 1` et sans `break`, le programme afficherait le résultat de l'addition puis celui de la soustraction, puis de la multiplication ! Ce comportement est parfois utilisé volontairement pour regrouper des cas (plusieurs `case` vides qui se suivent partagent le même bloc), mais c'est une source d'erreurs redoutable chez le débutant.



Arbre de décision d'un switch à 4 branches pour le menu de la calculatrice : chaque `break` ramène vers la suite du programme.

### 3.4 Les boucles : répéter sans se répéter

#### DÉFINITION Itération, compteur et accumulateur

Une **itération** (ou boucle) est une structure qui répète l'exécution d'un bloc d'instructions tant qu'une condition est satisfaite. Un **compteur** est une variable qui compte les passages dans la boucle (souvent nommée `i`). Un **accumulateur** est une variable qui cumule progressivement un résultat (somme, produit) : on l'initialise avant la boucle et on le met à jour à chaque tour.

**La boucle `for`.** On l'utilise quand le nombre de tours est connu à l'avance. Sa syntaxe regroupe les trois éléments de pilotage sur une seule ligne :

```
for ( initialisation ; condition ; increment ){
    // corps de la boucle
}
```

Exemple : la somme  $S = 1 + 2 + \dots + N$  des  $N$  premiers entiers, avec un accumulateur :

```
int N, i, somme = 0; // accumulateur initialise a 0
printf("Entrez N : ");
scanf("%d", &N);
for (i = 1; i <= N; i++){
    somme = somme + i; // accumulation
}
printf("Somme = %d\n", somme);
```

Pour  $N = 4$ , la variable `somme` vaut successivement 1, 3, 6, 10 : on obtient bien  $1 + 2 + 3 + 4 = 10$ .

**La boucle `while`.** On l'utilise quand on ne sait pas combien de tours seront nécessaires : on répète tant qu'une condition est vraie. Le test est fait avant chaque exécution du corps, qui peut donc ne jamais s'exécuter.

```
int i = 1;
while (i <= 5) {    // test AVANT le corps
    printf("%d", i);
    i++;            // fait évoluer la condition
}
```

**La boucle `do...while`.** Identique au `while`, mais le test est fait après le corps : celui-ci s'exécute donc **au moins une fois**. C'est la boucle idéale pour la saisie contrôlée.

**EXEMPLE** Saisie contrôlée d'une note entre 0 et 20, puis mention

```
#include <stdio.h>

int main(void) {
    float note;
    do {
        printf("Entrez une note entre 0 et 20 : ");
        scanf("%f", &note);
        if (note < 0 || note > 20) {
            printf("Note invalide, recommencez.\n");
        }
    } while (note < 0 || note > 20); // on recommence tant que c'est faux

    printf("Note validee : %.2f\n", note);
    if (note >= 16) {
        printf("Mention Tres Bien\n");
    } else if (note >= 14) {
        printf("Mention Bien\n");
    } else if (note >= 12) {
        printf("Mention Assez Bien\n");
    } else if (note >= 10) {
        printf("Mention Passable\n");
    } else {
        printf("Ajourné\n");
    }
    return 0;
}
```

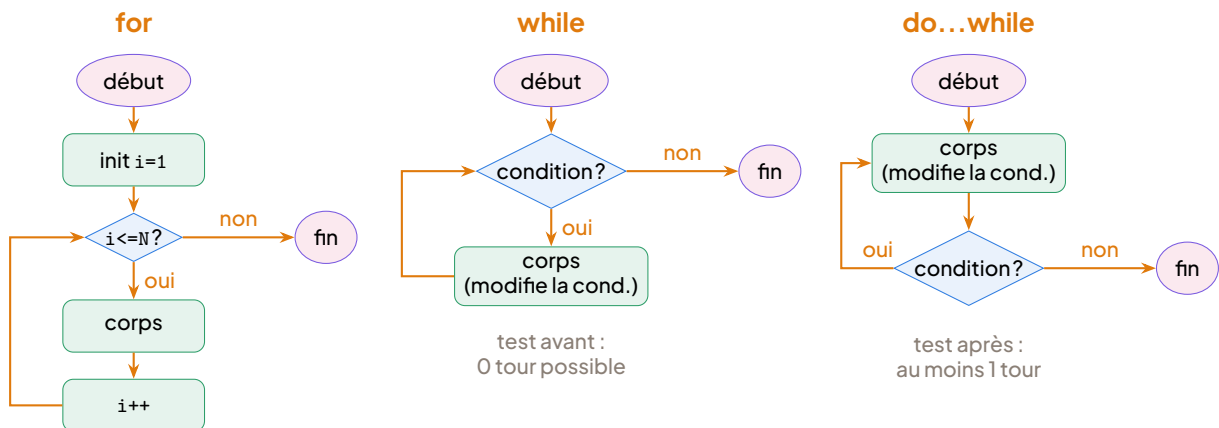
La question est posée une première fois, puis reposée tant que la valeur saisie est hors de l'intervalle  $[0; 20]$ . C'est le schéma type de toute saisie sécurisée.

**ATTENTION** La boucle infinie

Dans un `while`, si la condition ne dépend de rien qui évolue dans le corps, elle reste vraie éternellement :

```
int i = 1;
while (i <= 5) {
    printf("%d", i); // i n'est jamais modifié : boucle infinie !
}
```

Règle de survie : vérifie toujours qu'au moins une variable de la condition est modifiée dans le corps de la boucle, et qu'elle finira par rendre la condition fausse. Autre piège : `while (i <= 5);` avec un point-virgule crée un corps vide et boucle sans fin.



Organigrammes comparés des trois boucles : dans `for` et `while`, le test précède le corps; dans `do...while`, il le suit, ce qui garantit au moins une exécution.

### 3.5 Équivalence entre boucles et choix de la bonne boucle

**PROPRIÉTÉ** Toute boucle `for` peut se réécrire en `while`, et réciproquement

Construisons l'équivalence sur l'exemple de la somme des  $N$  premiers entiers. La version `for` :

```
for (i = 1; i <= N; i++) {
    somme = somme + i;
}
```

se décompose mécaniquement en trois étapes : l'initialisation est placée avant la boucle, la condition devient celle du `while`, et l'incrément est déplacé à la fin du corps :

```
i = 1; // 1. initialisation AVANT
while (i <= N) { // 2. condition dans le while
    somme = somme + i;
    i++; // 3. increment A LA FIN du corps
}
```

**MÉTHODE Choisir sa boucle**

1. Le nombre de tours est-il connu à l'avance (par exemple  $N$  élèves, les tables de 1 à 10)? → utilise `for`.
2. Sinon, la répétition dépend-elle d'une condition qui peut être fausse dès le départ (lire des données tant qu'on n'a pas atteint la fin)? → utilise `while`.
3. Le corps doit-il s'exécuter au moins une fois avant tout test (saisie contrôlée, menu à réafficher)? → utilise `do...while`.

**3.6 Compteurs, accumulateurs et boucles imbriquées**

**Factorielle itérative.** La factorielle de  $n$  est le produit  $n! = 1 \times 2 \times \dots \times n$ . L'accumulateur est ici un produit, donc initialisé à 1 (et non à 0!) :

```
int n, i;
long fact = 1; // accumulateur-produit initialise a 1
printf("Entrez n : ");
scanf("%d", &n);
for (i = 1; i <= n; i++){
    fact = fact * i;
}
printf("%d! = %ld\n", n, fact);
```

**Boucles imbriquées.** On place une boucle dans une autre pour parcourir des structures à deux dimensions : lignes et colonnes. La boucle interne tourne entièrement à chaque tour de la boucle externe.

```
// Table de multiplication de 1 a 5
int i, j;
for (i = 1; i <= 5; i++){           // boucle externe : les lignes
    for (j = 1; j <= 5; j++){       // boucle interne : les colonnes
        printf("%4d", i * j);      // %4d : affichage aligne sur 4 caracteres
    }
    printf("\n");                  // retour a la ligne apres chaque ligne
}
```

**Instructions `break` et `continue` (complément Bac).** Dans une boucle, `break` provoque la sortie immédiate de la boucle (utile pour arrêter une recherche dès que l'élément est trouvé); `continue` saute le reste du corps et passe directement au tour suivant. Par exemple, pour afficher les nombres de 1 à 10 sauf les multiples de 3 :

```
for (i = 1; i <= 10; i++){
    if (i % 3 == 0){
        continue; // on saute les multiples de 3
    }
    printf("%d ", i);
}
```

**EXERCICE RÉSOLU 3** Moyenne d'une classe avec saisie contrôlée

Énoncé. Écris un programme qui demande le nombre  $N$  d'élèves d'une classe de Terminale TI, fait saisir les  $N$  notes (chacune contrôlée entre 0 et 20), puis affiche la moyenne de la classe et le nombre d'élèves admis (note  $\geq 10$ ).

**SOLUTION**

Solution détaillée. Le nombre de notes est connu ( $N$ ) : boucle `for` externe. Chaque note doit être validée : boucle `do...while` interne. On utilise un accumulateur `somme` et un compteur `admis`.

```
#include <stdio.h>

int main(void){
    int N, i, admis = 0; // compteur d'admis
    float note, somme = 0.0f; // accumulateur

    printf("Nombre d'eleves : ");
    scanf("%d", &N);

    for (i = 1; i <= N; i++){
        do{
            printf("Note de l'eleve %d (0 a 20) : ", i);
            scanf("%f", &note);
        } while (note < 0 || note > 20);

        somme = somme + note; // accumulation
        if (note >= 10){
            admis++; // comptage
        }
    }

    printf("Moyenne de la classe : %.2f\n", somme / N);
    printf("Nombre d'admis : %d sur %d\n", admis, N);
    return 0;
}
```

Trace pour  $N = 3$  et les notes 12, 7, 15 : `somme` vaut successivement 12, 19, 34 ; `admis` vaut 1, 1, 2. Le programme affiche une moyenne de  $34/3 \approx 11,33$  et 2 admis sur 3. Remarque l'écriture `somme / N` : comme `somme` est un `float`, la division est bien réelle (pas de division entière).

**À RETENIR** L'essentiel de la section

- `if...else` exécute un bloc selon qu'une condition est vraie ou fausse ; un `else` se rattaché toujours au `if` le plus proche : accolades obligatoires pour éviter les pièges.
- `switch` compare une expression entière à des cas ; chaque `case` doit se terminer par `break` sous peine de fall-through.
- `for` : nombre de tours connu ; `while` : répétition conditionnelle (0 tour possible) ; `do...while` : au moins un tour (saisie contrôlée).
- Toute boucle `for` se réécrit en `while` (initialisation avant, incrément à la fin du corps).
- Compteur = variable qui compte ; accumulateur = variable qui cumule (somme initialisée à 0, produit initialisé à 1).
- Dans toute boucle, la condition doit évoluer dans le corps, sinon : boucle infinie.

**EXERCICE 3** À toi de jouer

1. Écris un programme qui affiche tous les nombres pairs entre 1 et 50, d'abord avec un `for`, puis avec un `while`.
2. Écris un programme qui demande un entier  $n$  et affiche sa table de multiplication de 1 à 12.
3. Réécris le menu de la calculatrice en utilisant un `do...while` pour que le menu se réaffiche tant que l'utilisateur ne choisit pas l'option « 5. Quitter ».
4. Écris un programme qui calcule et affiche la somme  $1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$  (attention à la division entière!).

Dans la prochaine section, nous apprendrons à découper nos programmes en fonctions réutilisables, à comprendre la portée des variables et à regrouper des données dans des enregistrements : de quoi structurer des applications de plus en plus ambitieuses.

## 4 FONCTIONS, PORTÉE DES VARIABLES ET ENREGISTREMENTS

Dans la section précédente, tu as appris à contrôler le déroulement d'un programme grâce aux structures alternatives (`if`, `switch`) et aux boucles (`for`, `while`, `do...while`). Tes programmes tiennent désormais dans un seul bloc `main()` qui peut devenir très long : imagine le programme de gestion des notes d'un lycée de Yaoundé, avec saisie, calculs de moyennes, classement, affichage des bulletins... tout cela dans une seule fonction de plusieurs centaines de lignes devient illisible, impossible à tester et à corriger. Les professionnels du logiciel résolvent ce

problème depuis toujours par un principe simple : diviser pour régner. On découpe le grand problème en petits sous-problèmes, et chaque sous-problème devient une **fonction**. C'est l'objet de cette section, qui te donnera aussi un outil pour regrouper des données de natures différentes : l'**enregistrement**.

## 4.1 Notion de fonction : prototype, définition, appel

### DÉFINITION Fonction

Une **fonction** est un bloc d'instructions nommé, qui reçoit éventuellement des données en entrée (les paramètres), effectue un traitement, et renvoie éventuellement un résultat (la valeur de retour). Une fonction peut être appelée depuis une autre fonction, autant de fois que nécessaire.

Tu connais déjà des fonctions : `printf`, `scanf` (bibliothèque `stdio.h`) et `sqrt`, `cos` (bibliothèque `math.h`). La grande nouveauté de cette section, c'est que tu vas écrire tes propres fonctions.

Une fonction en C se manifeste sous trois formes complémentaires :

- le **prototype** (ou déclaration) : il annonce au compilateur le nom de la fonction, son type de retour et les types de ses paramètres. Il se termine par un point-virgule ;
- la **définition** : elle contient le corps de la fonction, c'est-à-dire les instructions effectivement exécutées ;
- l'**appel** : c'est l'utilisation de la fonction, qui déclenche l'exécution de son corps.

### PROPRIÉTÉ Syntaxe du prototype et de la définition

```
// Prototype (place avant main, apres les #include)
type_retour nom_fonction(type1 param1, type2 param2);

// Definition
type_retour nom_fonction(type1 param1, type2 param2)
{
    // instructions
    return expression; // obligatoire si type_retour n'est pas void
}
```

Le type de retour peut être `int`, `float`, `char`, etc. Le mot-clé `return` termine immédiatement la fonction et renvoie la valeur à l'appelant.

### MÉTHODE Écrire une fonction en 4 étapes

1. Choisir le type de retour : que doit produire la fonction ? Un réel (float), un entier (int), rien du tout (void) ?
2. Nommer la fonction avec un verbe ou un nom explicite : calculerMoyenne, afficherMenu. Un bon nom rend le programme auto-documenté.
3. Lister les paramètres : de quelles données la fonction a-t-elle besoin pour travailler ? Préciser le type de chacune.
4. Écrire le corps : les instructions du traitement, terminées par `return` si la fonction renvoie une valeur. Ne pas oublier le prototype avant `main()`.

**EXEMPLE** Première fonction complète : la moyenne de deux notes

```
#include <stdio.h>

// Prototype : annonce de la fonction au compilateur
float moyenne(float a, float b);

int main(void)
{
    float n1, n2, m;
    printf("Note 1 : ");
    scanf("%f", &n1);
    printf("Note 2 : ");
    scanf("%f", &n2);
    m = moyenne(n1, n2); // appel : n1 et n2 sont les parametres effectifs
    printf("Moyenne = %.2f\n", m);
    return 0;
}

// Definition de la fonction
float moyenne(float a, float b) // a et b : parametres formels
{
    float resultat;
    resultat = (a + b) / 2;
    return resultat;          // on aurait pu ecrire : return (a+b)/2;
}
```

**DÉFINITION** Paramètres formels et effectifs

Les **paramètres formels** sont les variables déclarées dans l'en-tête de la fonction (a et b ci-dessus) : ce sont des noms de rôle. Les **paramètres effectifs** (ou arguments) sont les valeurs concrètes transmises lors de l'appel (n1 et n2). Lors de l'appel, chaque paramètre formel reçoit une copie du paramètre effectif correspondant, dans l'ordre.

**ATTENTION** Prototype manquant : l'erreur classique du débutant

Si tu appelles une fonction avant que le compilateur ne l'ait vue (ni prototype ni définition au-dessus de l'appel), tu obtiens une erreur ou un avertissement du type `implicit declaration of function`. Le compilateur ne connaît alors ni le type de retour ni les types des paramètres, et peut générer un programme faux. Règle d'or : toujours placer les prototypes juste après les `#include`, ou définir les fonctions avant `main()`. Vérifie aussi que le prototype et la définition ont exactement les mêmes types.

## 4.2 Transmission des paramètres par valeur

**PROPRIÉTÉ** Passage par valeur en C

En C, les paramètres sont toujours transmis par valeur : la fonction travaille sur une copie des arguments. Toute modification d'un paramètre à l'intérieur de la fonction n'a aucun effet sur la variable de l'appelant.

Illustrons cette propriété fondamentale, souvent mal comprise :

```
#include <stdio.h>

void doubler(int x); // prototype

int main(void)
{
    int n = 5;
    doubler(n);
    printf("n = %d\n", n); // affiche n = 5 : n n'a PAS change !
    return 0;
}

void doubler(int x)
{
    x = 2 * x; // modifie seulement la COPIE locale x
    printf("x = %d\n", x); // affiche x = 10
}
```

La variable `n` de `main` et le paramètre `x` de `doubler` sont deux cases mémoire différentes. Au moment de l'appel, la valeur 5 est recopiée dans `x`. Quand `doubler` termine, sa case `x` est détruite et `n` vaut toujours 5. Pour « renvoyer » un résultat, on utilise donc `return` (la transmission par adresse, avec les pointeurs, dépasse le programme de Terminale TI).

## 4.3 Variables locales, variables globales et pile d'appels

**DÉFINITION** Variable locale

Une **variable locale** est déclarée à l'intérieur d'une fonction (ou d'un bloc). Sa portée est limitée à cette fonction : elle est invisible ailleurs. Sa durée de vie correspond au temps d'exécution de la fonction : elle est créée à l'entrée et détruite à la sortie.

**DÉFINITION** Variable globale

Une **variable globale** est déclarée en dehors de toute fonction, généralement après les `#include`. Elle est accessible depuis toutes les fonctions du fichier et existe pendant toute la durée du programme.

**EXEMPLE** Portée locale contre globale

```
#include <stdio.h>

int compteur = 0; // variable GLOBALE : visible partout

void incrementer(void)
{
    int pas = 1;      // variable LOCALE a incrementer
    compteur = compteur + pas; // acces a la globale : autorise
}

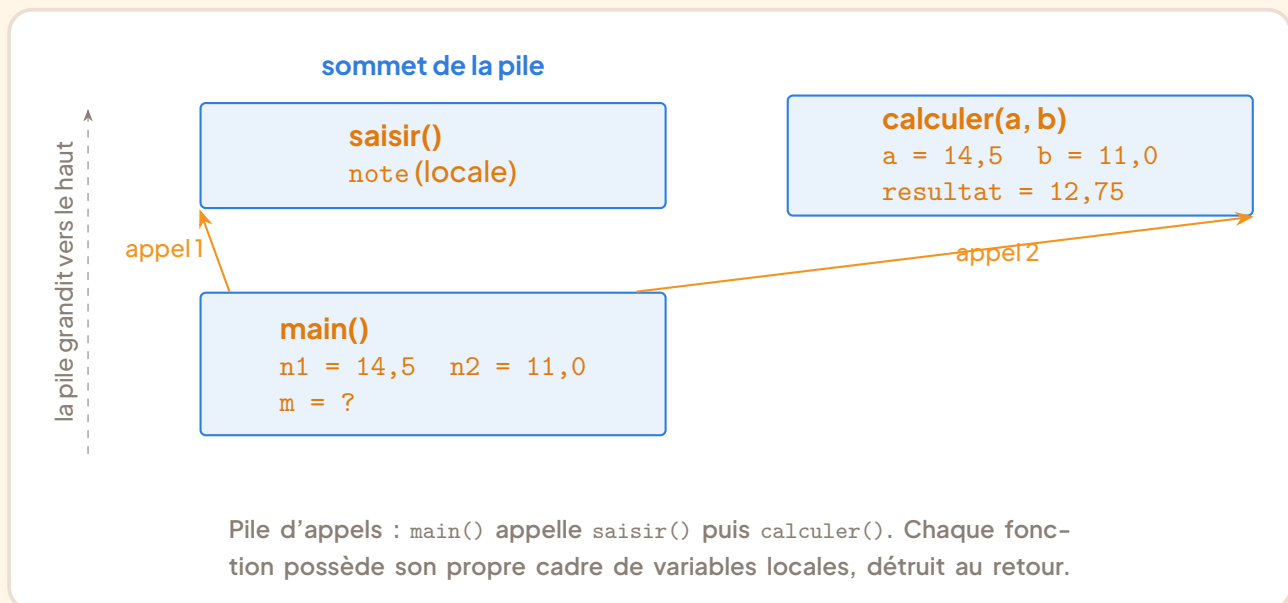
int main(void)
{
    incrementer();
    incrementer();
    printf("compteur = %d\n", compteur); // affiche 2
    // printf("%d", pas); // ERREUR de compilation : pas est inconnu ici
    return 0;
}
```

**ATTENTION** Les dangers des variables globales

Les globales semblent pratiques, mais elles rendent les programmes fragiles : n'importe quelle fonction peut les modifier, et quand une erreur apparaît, il est très difficile de savoir qui a changé la valeur. Deux fonctions qui utilisent la même globale deviennent impossibles à tester séparément. Règle professionnelle : privilégie les variables locales et les paramètres ; réserve les globales aux constantes partagées (par exemple un taux de TVA déclaré avec `const`).

Pour bien comprendre la mécanique des appels, il faut visualiser la **pile d'appels** (call stack) :

à chaque appel de fonction, un cadre contenant ses paramètres et ses variables locales est empilé; au `return`, le cadre est dépilé et l'exécution reprend dans l'appelant.



#### 4.4 Fonctions void et modularisation : l'exemple de la calculatrice

Une fonction **sans valeur de retour** est déclarée avec le type `void` : elle effectue une action (affichage, saisie) mais ne renvoie rien; on y écrit `return`; ou rien du tout. Une fonction **sans paramètre** s'écrit avec des parenthèses contenant `void`: `void afficherMenu(void)`.

Décomposons une calculatrice en fonctions, chacune faisant une seule chose :

```
#include <stdio.h>

/* ---- Prototypes ---- */
float additionner(float a, float b);
float soustraire(float a, float b);
void afficherMenu(void);

int main(void)
{
    int choix;
    float x, y;
    afficherMenu();           // fonction sans parametre, sans retour
    scanf("%d", &choix);
    printf("Deux nombres : ");
    scanf("%f %f", &x, &y);
    if (choix == 1)
        printf("Somme = %.2f\n", additionner(x, y));
    else if (choix == 2)
        printf("Difference = %.2f\n", soustraire(x, y));
    else
        printf("Choix invalide.\n");
    return 0;
}

/* ---- Definitions ---- */
void afficherMenu(void)
```

```
{
    printf("=== CALCULATRICE ===\n");
    printf("1. Addition\n2. Soustraction\nVotre choix : ");
}

float additionner(float a, float b)
{
    return a + b;
}

float soustraire(float a, float b)
{
    return a - b;
}
```

Le `main()` devient une sorte de « chef d'orchestre » très lisible : il appelle des fonctions dont le nom dit exactement ce qu'elles font. C'est la **modularisation**.

#### LESAVAIS-TU ? Une fonction = une seule tâche, bien faite

Découper un programme en fonctions est le premier pas vers la programmation professionnelle. Le principe de « responsabilité unique » — chaque fonction fait une seule chose et la fait bien — est au cœur du génie logiciel. Dans les startups tech de Douala ou de Yaoundé qui développent des applications de paiement mobile, le code est organisé en milliers de petites fonctions testées individuellement : quand un bug apparaît, on sait immédiatement où chercher, et une fonction corrigée profite à tout le programme.

#### EXERCICE RÉSOLU 4 Moyennes de trois élèves

Énoncé : écrire un programme qui demande les deux notes de trois élèves et affiche la moyenne de chacun, en utilisant une fonction `float moyenne(float a, float b)`.

#### SOLUTION

Solution détaillée : la fonction est écrite une seule fois, puis appelée trois fois dans une boucle — c'est tout l'intérêt de la modularisation.

```
#include <stdio.h>

float moyenne(float a, float b); // prototype

int main(void)
{
    int i;
    float n1, n2;
    for (i = 1; i <= 3; i++)
    {
        printf("Eleve %d - note 1 : ", i);
        scanf("%f", &n1);
        printf("Eleve %d - note 2 : ", i);
```

```
scanf("%f", &n2);
printf("-> Moyenne de l'eleve %d : %.2f\n",
      i, moyenne(n1, n2)); // appel dans printf
}
return 0;
}

float moyenne(float a, float b)
{
    return (a + b) / 2;
}
```

Trace chiffrée : pour l'élève 1 avec  $n1 = 12$  et  $n2 = 16$ , l'appel copie 12 dans  $a$  et 16 dans  $b$ ; la fonction renvoie  $(12+16)/2 = 14$ . Pour l'élève 2 avec 8 et 13, elle renvoie 10,5; pour l'élève 3 avec 15 et 9, elle renvoie 12. Sans fonction, il aurait fallu recopier la formule trois fois — et la corriger trois fois en cas d'erreur.

#### 4.5 Les enregistrements : la structure `struct`

Jusqu'ici, une variable ne contient qu'une seule valeur d'un seul type. Or une fiche élève réunit des informations de types différents : un nom (chaîne), une moyenne (réel), un âge (entier). Le C permet de regrouper ces données dans un **enregistrement**.

##### DÉFINITION Enregistrement (struct)

Un **enregistrement** est un type de données composé, défini avec le mot-clé `struct`, qui regroupe plusieurs variables appelées champs, chacune avec son propre type. On accède à chaque champ avec l'opérateur point : `variable.champ`.

##### PROPRIÉTÉ Syntaxe de déclaration et d'utilisation

```
// Declaration du type (apres les #include, avant les fonctions)
struct Eleve
{
    char nom[30];
    float moyenne;
    int age;
};

// Dans une fonction : declaration d'une variable de ce type
struct Eleve e1;

// Acces aux champs avec l'operateur point
e1.age = 17;
e1.moyenne = 13.75;
```

Le point-virgule après l'accolade fermante de la `struct` est obligatoire : son oubli est une erreur de compilation fréquente.

```
struct Eleve e1;
```

| nom                   | moyenne          | age           |
|-----------------------|------------------|---------------|
| « Amina » (30 octets) | 13,75 (4 octets) | 17 (4 octets) |

une seule variable, trois champs juxtaposés en mémoire

Un enregistrement `struct Eleve` en mémoire : les champs `nom`, `moyenne` et `age` occupent des cases juxtaposées dans une seule variable.

#### EXEMPLE Saisie et affichage d'une fiche élève

```
#include <stdio.h>

struct Eleve
{
    char nom[30];
    float moyenne;
    int age;
};

int main(void)
{
    struct Eleve e;
    printf("Nom de l'eleve : ");
    scanf("%s", e.nom);    // pas de & : nom est déjà un tableau
    printf("Age : ");
    scanf("%d", &e.age);   // & obligatoire pour un int
    printf("Moyenne : ");
    scanf("%f", &e.moyenne); // & obligatoire pour un float

    printf("\n--- FICHE ELEVE ---\n");
    printf("Nom      : %s\n", e.nom);
    printf("Age       : %d ans\n", e.age);
    printf("Moyenne   : %.2f / 20\n", e.moyenne);
    return 0;
}
```

#### ATTENTION Pièges classiques avec les struct

- Oublier le ; après l'accolade fermante de la déclaration : erreur de compilation parfois

difficile à localiser.

- Écrire `scanf("%s", &e.nom)` : pour une chaîne (tableau de `char`), il ne faut pas de `&`, car le nom d'un tableau désigne déjà son adresse. En revanche le `&` reste obligatoire pour les champs `int` et `float`.
- Confondre le type (`struct Eleve`) et la variable (`e`) : c'est la variable qui porte les valeurs, via le point.
- `scanf("%s")` s'arrête au premier espace. Pour un nom composé (ex. « Jean Pierre »), utilise `fgets(e.nom, 30, stdin)` et retire le `\n` final.

#### 4.6 Algorithme intégrant un enregistrement : la tontine

Mettons tout ensemble — enregistrement, boucle, fonction — dans une application réaliste : la gestion simplifiée des membres d'une tontine, cette association d'épargne rotative très répandue au Cameroun. Chaque membre est décrit par son nom et sa cotisation mensuelle.

##### EXPÉRIENCE TP : gestion des membres d'une tontine

Objectif : saisir les membres d'une tontine, calculer le total des cotisations et afficher la liste.

Logiciels : Code ::Blocks (ou Dev-C++), compilateur GCC.

Manipulation : crée un projet console en C, saisis le programme ci-dessous, compile (F9) puis exécute pas à pas (F8 dans Code ::Blocks) en observant les champs de la variable `m` dans la fenêtre des variables.

```
#include <stdio.h>

#define NB_MEMBRES 3 // constante : nombre de membres

struct Membre
{
    char nom[30];
    float cotisation; // en FCFA
};

float totalCotisations(struct Membre m); // prototype

int main(void)
{
    struct Membre m;
    float total = 0;
    int i;

    for (i = 1; i <= NB_MEMBRES; i++)
    {
        printf("Membre %d - nom : ", i);
        scanf("%s", m.nom);
        printf("Membre %d - cotisation (FCFA) : ", i);
```

```
        scanf("%f", &m.cotisation);
        total = total + totalCotisations(m);
    }
    printf("\nTotal des cotisations : %.0f FCFA\n", total);
    return 0;
}

float totalCotisations(struct Membre m)
{
    printf("Enregistre : %s(%.0f FCFA)\n", m.nom, m.cotisation);
    return m.cotisation;
}
```

Observations : l'enregistrement `m` est transmis à la fonction par valeur : la fonction reçoit une copie complète de la fiche. En exécution pas à pas, on voit les champs `m.nom` et `m.cotisation` se remplir à chaque tour de boucle.

Interprétation : regrouper `nom` et `cotisation` dans une `struct` garantit que les deux informations d'un même membre voyagent toujours ensemble — impossible de mélanger le nom de l'un avec la cotisation de l'autre.

#### À RETENIR L'essentiel de la section

- Une fonction se déclare par un **prototype**, se définit avec un corps, et s'utilise par un **appel**. Le mot-clé `return` renvoie le résultat ; `void` signifie « pas de retour » ou « pas de paramètre ».
- En C, les paramètres sont transmis **par valeur** : la fonction travaille sur une copie et ne peut pas modifier les variables de l'appelant.
- Une variable **locale** n'existe que dans sa fonction ; une variable **globale** est visible partout mais doit être évitée (sauf constantes).
- Une **struct** regroupe des champs de types différents ; on accède à un champ par l'opérateur point : `e.moyenne`.
- Toujours placer les prototypes après les `#include` pour éviter l'erreur d'« implicit declaration ».

#### EXERCICE 4 À toi de jouer

1. Écris une fonction `float surfaceRectangle(float L, float l)` puis un `main()` qui l'utilise pour calculer la surface d'une parcelle à Bafoussam.
2. Que s'affiche-t-il si l'on supprime le prototype de `moyenne` dans l'exercice résolu ? Expérimente et explique le message du compilateur.

3. Définis une `struct Produit` (nom, prix, quantité) et écris un programme qui saisit un produit et affiche la valeur du stock (prix × quantité).
4. Reprends le TP de la tontine : ajoute une fonction `void afficherMembre(struct Membre m)` qui affiche proprement une fiche, et appelle-la dans la boucle.

**CORRIGÉ Exercice n°1**

```
#include <stdio.h>

float surfaceRectangle(float L, float l);

int main(void)
{
    float longueur, largeur;
    printf("Longueur et largeur (m) : ");
    scanf("%f %f", &longueur, &largeur);
    printf("Surface = %.2f m2\n",
        surfaceRectangle(longueur, largeur));
    return 0;
}

float surfaceRectangle(float L, float l)
{
    return L * l;
}
```

La fonction reçoit des copies de `longueur` et `largeur` (passage par valeur) et renvoie le produit, affiché avec deux décimales grâce à `%.2f`.

**CORRIGÉ Exercice n°2**

Sans prototype, le compilateur émet un avertissement : `warning: implicit declaration of function 'moyenne'`. Il suppose alors que la fonction retourne un `int` et accepte n'importe quels arguments. Comme `moyenne` retourne un `float`, la valeur récupérée dans `main` est corrompue (interprétation binaire erronée), ce qui affiche un résultat absurde (souvent 0.00 ou une valeur géante). Le programme compile (avec avertissement) mais s'exécute mal. Moralité : ne jamais ignorer les avertissements de prototype.

**CORRIGÉ Exercice n°3**

```
#include <stdio.h>
```

```
struct Produit
{
    char nom[30];
    float prix;
    int quantite;
};

int main(void)
{
    struct Produit p;
    printf("Nom du produit : ");
    scanf("%s", p.nom);
    printf("Prix unitaire (FCFA) : ");
    scanf("%f", &p.prix);
    printf("Quantite en stock : ");
    scanf("%d", &p.quantite);
    printf("Valeur du stock de %s : %.0f FCFA\n",
        p.nom, p.prix * p.quantite);
    return 0;
}
```

Attention au & : obligatoire pour p.prix et p.quantite, interdit devant p.nom qui est un tableau de caractères.

#### CORRIGÉ Exercice n°4

```
#include <stdio.h>

#define NB_MEMBRES 3

struct Membre
{
    char nom[30];
    float cotisation;
};

void afficherMembre(struct Membre m); // prototype

int main(void)
{
    struct Membre m;
    float total = 0;
    int i;

    for (i = 1; i <= NB_MEMBRES; i++)
    {
        printf("Membre %d - nom : ", i);
        scanf("%s", m.nom);
        printf("Membre %d - cotisation (FCFA) : ", i);
        scanf("%f", &m.cotisation);
        afficherMembre(m); // appel de la nouvelle fonction
        total = total + m.cotisation;
    }
}
```

```
printf("\nTotal des cotisations : %.0f FCFA\n", total);
return 0;
}

void afficherMembre(struct Membre m)
{
    printf("  -> Fiche : %s | Cotisation : %.0f FCFA\n", m.nom, m.cotisation);
}
```

La fonction `afficherMembre` reçoit une copie de la structure (passage par valeur), ce qui est suffisant pour un affichage en lecture seule.

Tu sais désormais découper un programme en fonctions claires et structurer tes données avec des enregistrements. Dans la prochaine section, nous appliquerons ces acquis aux algorithmes classiques sur les tableaux : la recherche séquentielle et le tri par sélection.

## 5 ALGORITHMES CLASSIQUES : RECHERCHE SÉQUENTIELLE ET TRI PAR SÉLECTION

Dans la section précédente, tu as appris à découper un programme en **fonctions**, à maîtriser la **portée** des variables (locales et globales) et à regrouper des données hétérogènes dans un **enregistrement** (`struct`). Nous allons maintenant mettre ces acquis au service de deux problèmes extrêmement concrets, que rencontre tout développeur camerounais qui écrit une application monoposte : retrouver une information dans une liste (par exemple le nom d'un élève dans le tableau des inscrits d'un lycée de Yaoundé) et ranger des données dans l'ordre (par exemple classer les cotisants d'une tontine par montant). Ces deux problèmes se résolvent par deux algorithmes classiques au programme officiel : la **recherche séquentielle** et le **tri par sélection**.

### 5.1 Rappel : les tableaux en langage C

Avant de chercher ou de trier, il faut stocker les données. En C, un **tableau** est une suite de cases mémoire de même type, accessibles par un **indice**.

#### PROPRIÉTÉ Déclaration et indices d'un tableau en C

La déclaration `int T[50];` réserve 50 cases entières nommées `T[0]`, `T[1]`, ..., `T[49]`. Le premier indice est toujours 0 et le dernier toujours `N-1` pour un tableau de `N` cases. La saisie et l'affichage se font par une boucle `for`.

#### EXEMPLE Saisie et affichage des notes d'une classe

```
#include <stdio.h>

int main(void){
    int T[50]; /* tableau pouvant contenir jusqu'à 50 notes */
    int N, i;

    printf("Nombre de notes (max 50) : ");
    scanf("%d", &N);

    /* Saisie : on remplit les cases 0 à N-1 */
    for (i = 0; i < N; i++){
        printf("Note numéro %d : ", i+1);
        scanf("%d", &T[i]);
    }

    /* Affichage : on relit les cases 0 à N-1 */
    printf("Notes saisies : ");
    for (i = 0; i < N; i++){
        printf("%d ", T[i]);
    }
    printf("\n");
    return 0;
}
```

**ATTENTION** Ne jamais dépasser l'indice  $N-1$ 

L'erreur la plus fréquente (et la plus dangereuse) est d'écrire  $T[N]$  ou de faire tourner une boucle avec la condition  $i \leq N$ . Le C ne vérifie pas les bornes : lire ou écrire  $T[N]$  accède à une zone mémoire qui n'appartient pas au tableau. Le programme peut alors afficher des valeurs absurdes, modifier d'autres variables, ou planter de façon imprévisible. Retiens la règle d'or : boucle `for (i = 0; i < N; i++)`, jamais  $\leq N$ .

## 5.2 La recherche séquentielle

**Intuition.** Tu cherches la fiche d'un élève nommé « Kamga » dans un paquet de fiches non rangées. Il n'y a qu'une seule méthode : prendre les fiches une par une, dans l'ordre, jusqu'à tomber sur la bonne — ou constater, arrivé au bout du paquet, qu'elle n'y est pas. C'est exactement la recherche séquentielle.

**DÉFINITION** Recherche séquentielle

La **recherche séquentielle** (ou recherche linéaire) d'une valeur  $x$  dans un tableau  $T$  de  $N$  éléments consiste à comparer  $x$  successivement à  $T[0]$ ,  $T[1]$ , ...,  $T[N-1]$ , jusqu'à trouver une case égale à  $x$  (on retourne alors son indice) ou épuiser le tableau (la valeur est absente).

**Démarche pas à pas :**

1. Initialiser un indice  $i$  à 0 et un booléen `trouve` à faux.
2. Tant que  $i < N$  et que  $T[i]$  est différent de  $x$ , avancer :  $i \leftarrow i + 1$ .
3. À la sortie de la boucle, si  $i < N$ , la valeur est trouvée à l'indice  $i$ ; sinon elle est absente.

**EXEMPLE** Fonction de recherche séquentielle complète

```
#include <stdio.h>

/* Retourne l'indice de la première occurrence de x dans T,
ou -1 si x est absent. */
int rechercheSequentielle(int T[], int N, int x) {
    int i = 0;
    while (i < N && T[i] != x) {
        i++; /* on avance tant qu'on n'a pas trouvé */
    }
    if (i < N) {
        return i; /* trouvé à l'indice i */
    } else {
        return -1; /* valeur absente : convention -1 */
    }
}

int main(void) {
    int notes[6] = {12, 5, 9, 3, 7, 14};
    int pos = rechercheSequentielle(notes, 6, 9);
    if (pos != -1) {
        printf("Valeur trouvée à l'indice %d\n", pos);
    } else {
        printf("Valeur absente du tableau\n");
    }
    return 0;
}
```

**ATTENTION** Deux pièges classiques

- Dans la condition `while (i < N && T[i] != x)`, le test  $i < N$  doit figurer en premier : grâce à l'évaluation paresseuse du `&&`, si  $i$  atteint  $N$ , le second test n'est pas évalué et on ne lit jamais  $T[N]$ .
- Ne confonds pas « retourner l'indice » et « retourner la valeur » : la fonction renvoie la position  $i$ , pas  $T[i]$ .

**PROPRIÉTÉ** Coût de la recherche séquentielle

Si la valeur cherchée est absente (ou se trouve en dernière case), la recherche séquentielle effectue exactement  $N$  comparaisons. Dans tous les cas, elle en effectue au plus  $N$  : on dit que son coût est **linéaire** en  $N$ . Ce résultat est admis, mais tu peux le vérifier expérimentalement en ajoutant un compteur incrémenté à chaque tour de boucle et en l'affichant.

**5.3 Le tri par sélection**

**Intuition.** Le proviseur veut afficher les notes d'une classe de la plus petite à la plus grande. Une méthode naturelle : parcourir toute la liste, repérer la plus petite note et la placer en première position ; puis, parmi les notes restantes, repérer la plus petite et la placer en deuxième position ; et ainsi de suite. À chaque tour, la zone déjà triée grandit d'une case.

**DÉFINITION** Tri par sélection

Le **tri par sélection** d'un tableau de  $N$  éléments consiste, pour chaque position  $i$  allant de 0 à  $N - 2$ , à :

1. rechercher l'**indice du minimum** dans la sous-partie non triée  $T[i..N-1]$  ;
2. échanger le contenu de  $T[i]$  avec celui de cette case minimale.

Après le tour  $i$ , les cases  $T[0..i]$  contiennent les  $i + 1$  plus petits éléments, dans l'ordre croissant : c'est la zone triée.

**DÉFINITION** Indice du minimum

L'**indice du minimum** d'une sous-partie  $T[i..N-1]$  est la position  $i_{\min}$  telle que  $T[i_{\min}]$  est le plus petit élément de cette sous-partie. On le calcule en initialisant  $i_{\min} = i$ , puis en parcourant  $j$  de  $i + 1$  à  $N - 1$  : chaque fois que  $T[j] < T[i_{\min}]$ , on met à jour  $i_{\min} = j$ .

**MÉTHODE** Échanger deux cases : la variable temporaire est obligatoire

Pour échanger  $T[i]$  et  $T[j]$ , il faut toujours une variable temporaire, sinon l'une des deux valeurs est écrasée et perdue :

```
int temp = T[i]; /* 1. on sauvegarde T[i] */
T[i] = T[j];     /* 2. on copie T[j] dans T[i] */
T[j] = temp;     /* 3. on restaure l'ancien T[i] dans T[j] */
```

Écrire directement  $T[i] = T[j]$  ;  $T[j] = T[i]$  ; est un bug classique : après la première

affectation, l'ancienne valeur de  $T[i]$  est définitivement perdue.

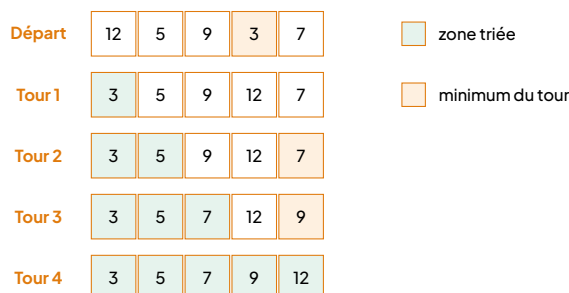
**EXEMPLE** Code C complet du tri par sélection

```
#include <stdio.h>

/* Tri par sélection : range T[0..N-1] en ordre croissant */
void triSelection(int T[], int N) {
    int i, j, imin, temp;
    for (i = 0; i < N - 1; i++) {          /* N-1 tours suffisent */
        /* 1. recherche de l'indice du minimum dans T[i..N-1] */
        imin = i;
        for (j = i + 1; j < N; j++) {
            if (T[j] < T[imin]) {
                imin = j;
            }
        }
        /* 2. échange de T[i] et T[imin] via variable temporaire */
        temp = T[i];
        T[i] = T[imin];
        T[imin] = temp;
    }
}

int main(void) {
    int notes[5] = {12, 5, 9, 3, 7};
    int i;
    triSelection(notes, 5);
    printf("Tableau trié : ");
    for (i = 0; i < 5; i++) {
        printf("%d ", notes[i]); /* affiche : 3 5 7 9 12 */
    }
    printf("\n");
    return 0;
}
```

La figure suivante illustre le déroulement du tri sur le tableau [12, 5, 9, 3, 7] : la zone coloriée en vert est la zone déjà triée, la case orange est le minimum repéré à ce tour.



Tri par sélection de [12, 5, 9, 3, 7] : à chaque tour, le minimum de la zone non triée (orange) est échangé avec la première case de cette zone, qui rejoint la zone triée (vert).

#### EXERCICE RÉSOLU 5 Trace complète du tri de [12, 5, 9, 3, 7]

Énoncé : effectuer la trace manuelle du tri par sélection du tableau [12, 5, 9, 3, 7] et dénombrer les comparaisons.

#### SOLUTION

Tour 1 ( $i = 0$ ) : on cherche le minimum de [12, 5, 9, 3, 7]. Comparaisons :  $5 < 12$  (oui,  $i_{\min} = 1$ ),  $9 < 5$  (non),  $3 < 5$  (oui,  $i_{\min} = 3$ ),  $7 < 3$  (non) : 4 comparaisons. Échange de  $T[0]$  et  $T[3]$  : [3, 5, 9, 12, 7].

Tour 2 ( $i = 1$ ) : minimum de [5, 9, 12, 7]. Comparaisons :  $9 < 5$  (non),  $12 < 5$  (non),  $7 < 5$  (non) : 3 comparaisons.  $i_{\min} = 1 = i$ , l'échange laisse le tableau inchangé : [3, 5, 9, 12, 7].

Tour 3 ( $i = 2$ ) : minimum de [9, 12, 7]. Comparaisons :  $12 < 9$  (non),  $7 < 9$  (oui,  $i_{\min} = 4$ ) : 2 comparaisons. Échange de  $T[2]$  et  $T[4]$  : [3, 5, 7, 12, 9].

Tour 4 ( $i = 3$ ) : minimum de [12, 9]. Comparaison :  $9 < 12$  (oui,  $i_{\min} = 4$ ) : 1 comparaison. Échange : [3, 5, 7, 9, 12].

Total :  $4 + 3 + 2 + 1 = 10$  comparaisons, et le tableau est trié. On remarque qu'aucun tour n'est nécessaire pour la dernière case : quand les  $N-1$  premières sont en place, la dernière l'est forcément.

### 5.4 Combien de comparaisons ? Le coût quadratique du tri par sélection

Comptons les comparaisons dans le cas général d'un tableau de  $N$  éléments. Au tour  $i$ , la recherche du minimum dans la sous-partie  $T[i..N-1]$ , qui contient  $N-i$  cases, exige  $N-i-1$  comparaisons. Le nombre total de comparaisons est donc :

$$C(N) = (N-1) + (N-2) + \dots + 2 + 1 = \sum_{k=1}^{N-1} k.$$

**PROPRIÉTÉ** Le tri par sélection effectue exactement  $\frac{N(N-1)}{2}$  comparaisons

Démonstration guidée. Notons  $S = 1 + 2 + \dots + (N-1)$ . Écrivons la somme dans les deux sens et additionnons membre à membre :

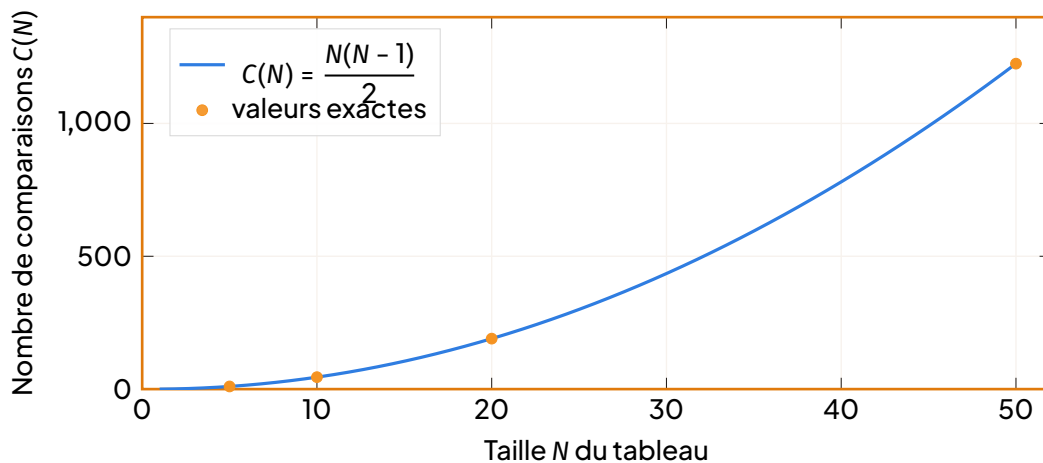
$$\begin{array}{r} S = 1 + 2 + \dots + (N-2) + (N-1) \\ S = (N-1) + (N-2) + \dots + 2 + 1 \\ \hline 2S = \underbrace{N + N + \dots + N + N}_{N-1 \text{ termes}} = N(N-1) \end{array}$$

car chaque colonne verticale totalise  $N$  (par exemple  $1 + (N-1) = N$ ,  $2 + (N-2) = N$ , etc.) et il y a  $N-1$  colonnes. Donc :

$$S = \frac{N(N-1)}{2}.$$

Le tri par sélection effectue donc exactement  $\frac{N(N-1)}{2}$  comparaisons, quel que soit l'ordre initial du tableau. Vérification avec notre exemple :  $N = 5$  donne  $\frac{5 \times 4}{2} = 10$  comparaisons, conforme à la trace.

On dit que le tri par sélection a un coût **quadratique** : quand  $N$  double, le nombre de comparaisons est multiplié par environ 4. Le graphique suivant montre cette croissance.



Croissance quadratique du nombre de comparaisons du tri par sélection : pour  $N = 50$ , on atteint déjà 1225 comparaisons ; pour  $N = 1000$ , près de 500 000.

**LESAVAIS-TU ?** Et pour les très grands tableaux ?

Le tri par sélection est le plus simple à comprendre et à programmer, mais son coût quadra-

tique devient rédhibitoire pour de très grands volumes : trier un million de fiches demanderait environ  $5 \times 10^{11}$  comparaisons. Les informaticiens utilisent alors des tris plus rapides comme le **tri rapide** (quicksort, inventé par Tony Hoare en 1960), dont le coût moyen est de l'ordre de  $N \log_2 N$  — soit environ 20 millions d'opérations pour un million de fiches, c'est-à-dire 25 000 fois moins. C'est ce genre d'algorithme qui permet aux applications de Mobile Money de traiter des millions de transactions. Au baccalauréat, cependant, c'est bien le tri par sélection qui est exigible : maîtrise-le parfaitement.

### 5.5 Application : classer les cotisants d'une tontine par montant décroissant

Une tontine de quartier à Douala veut afficher ses cotisants du plus gros contributeur au plus modeste. Il suffit d'inverser le sens de la comparaison : au lieu de chercher le minimum, on cherche le **maximum** de la zone non triée. On en profite pour utiliser un enregistrement (section précédente) afin de trier les montants sans perdre les noms.

#### EXEMPLE Tri décroissant des cotisants (enregistrements)

```
#include <stdio.h>
#include <string.h>

typedef struct {
    char nom[30];
    float montant; /* cotisation en FCFA */
} Cotisant;

/* Tri par sélection adapté : ordre DÉCROISSANT sur le montant */
void triCotisants(Cotisant T[], int N) {
    int i, j, imax;
    Cotisant temp;
    for (i = 0; i < N - 1; i++) {
        imax = i; /* on cherche le MAXIMUM */
        for (j = i + 1; j < N; j++) {
            if (T[j].montant > T[imax].montant) { /* > au lieu de < */
                imax = j;
            }
        }
        temp = T[i]; /* échange d'enregistrements complets */
        T[i] = T[imax];
        T[imax] = temp;
    }
}

int main(void) {
    Cotisant tontine[4] = {
        {"Mballa", 50000}, {"Ngo Bell", 120000},
        {"Etoundi", 75000}, {"Fotso", 200000}
    };
    int i;
    triCotisants(tontine, 4);
    printf("Classement des cotisants :\n");
}
```

```
for (i = 0; i < 4; i++){  
    printf("%d. %-10s : %.0fFCFA\n",  
        i + 1, tontine[i].nom, tontine[i].montant);  
}  
return 0;  
}
```

Affichage obtenu : Fotso (200 000), Ngo Bell (120 000), Etoundi (75 000), Mballa (50 000). Remarque que l'échange porte sur l'enregistrement entier : le nom suit toujours son montant.

#### À RETENIR L'essentiel de la section

- Un tableau `int T[N]` s'indice de 0 à  $N-1$ ; toute boucle de parcours s'écrit `for (i = 0; i < N; i++)`.
- La **recherche séquentielle** compare la valeur cherchée à chaque case dans l'ordre : au plus  $N$  comparaisons (coût linéaire); on retourne l'indice trouvé ou  $-1$  si la valeur est absente.
- Le **tri par sélection** place à chaque tour le minimum de la zone non triée en tête de cette zone, par échange avec variable temporaire.
- Il effectue exactement  $\frac{N(N-1)}{2}$  comparaisons (somme des entiers de 1 à  $N-1$ ) : coût quadratique, indépendant de l'ordre initial.
- Pour trier en ordre décroissant, on remplace la recherche du minimum par celle du maximum ( $>$  au lieu de  $<$ ).

#### EXERCICE 5 À toi de jouer

1. Écris en C une fonction `int occurrences(int T[], int N, int x)` qui retourne le nombre de fois où la valeur  $x$  apparaît dans le tableau (adaptation de la recherche séquentielle).
2. Modifie le tri par sélection pour qu'il trie un tableau de chaînes de caractères (noms d'élèves) par ordre alphabétique, en utilisant `strcmp` de `string.h`.
3. Combien de comparaisons le tri par sélection effectue-t-il sur un tableau de 200 élèves? Sur un tableau déjà trié de 200 élèves? Justifie.

**CORRIGÉ** Exercice 1

1. Parcourir tout le tableau avec un compteur `cpt` initialisé à 0 ; incrémenter à chaque égalité `T[i] == x` ; retourner `cpt`. Attention : ici on ne s'arrête pas au premier élément trouvé, la boucle va toujours jusqu'à  $N - 1$ .
2. Remplacer la comparaison `T[j] < T[imin]` par `strcmp(T[j], T[imin]) < 0` et l'échange par des copies avec `strcpy` via une chaîne temporaire `char temp[50]`.
3.  $C(200) = \frac{200 \times 199}{2} = 19\,900$  comparaisons, y compris si le tableau est déjà trié : le tri par sélection ne teste jamais si le tableau est en ordre, il effectue toujours toutes ses boucles. C'est une différence majeure avec certains tris adaptatifs.

## 6 RÉCURSIVITÉ, GESTION DES ERREURS ET PROJET D'APPLICATION COMPLÈTE

Dans la section précédente, nous avons résolu deux grands problèmes classiques — la recherche séquentielle et le tri par sélection — à l'aide de boucles. Mais il existe une autre manière, plus élégante, d'exprimer certains algorithmes : au lieu de répéter un traitement, on le décompose en un problème plus petit de même nature. C'est l'idée de la **récursivité**. Nous verrons ensuite comment lire et corriger les erreurs signalées par le compilateur, puis nous terminerons par deux projets complets qui mobilisent tout ce que tu as appris dans cette leçon.

### 6.1 La notion de fonction récursive

**Intuition : les poupées russes.** Imagine une tontine de quartier à Yaoundé : pour connaître le total collecté, le président demande au trésorier, qui demande à son adjoint, et ainsi de suite jusqu'au dernier membre qui, lui, connaît sa cotisation. Chacun ajoute sa part au résultat remonté par le suivant. Le problème « total de  $n$  membres » a été ramené au problème « total de  $n - 1$  membres », jusqu'à un cas si petit qu'on le résout directement. C'est exactement le mécanisme d'une fonction récursive.

**DÉFINITION** Fonction récursive, cas de base, cas récursif

Une **fonction récursive** est une fonction qui s'appelle elle-même dans son propre corps. Toute fonction récursive correcte comporte deux parties :

- le **cas de base** (ou condition d'arrêt) : un cas trivial résolu sans nouvel appel récursif ;
- le **cas récursif** : la fonction s'appelle elle-même sur un problème strictement plus petit, qui se rapproche du cas de base.

Si le cas de base est absent ou jamais atteint, les appels s'empilent sans fin jusqu'à épuiser la mémoire de la pile : c'est le **débordement de pile** (stack overflow), qui fait planter le programme.

**L'exemple fondateur : la factorielle.** En mathématiques,  $n! = n \times (n - 1) \times \dots \times 2 \times 1$ , avec  $0! = 1$ . On peut la définir autrement :

$$\text{fact}(0) = 1 \quad \text{et} \quad \text{fact}(n) = n \times \text{fact}(n - 1) \quad \text{pour } n \geq 1.$$

Cette définition se traduit en C mot pour mot :

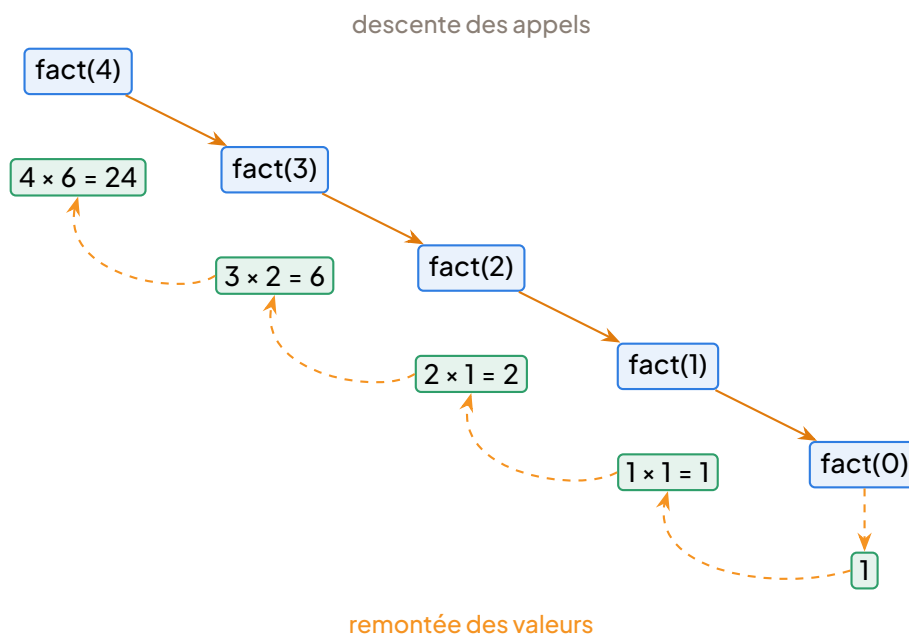
```
long fact(int n) {  
    if (n == 0)          /* cas de base : on s'arrete ici */  
        return 1;  
    else                 /* cas recursif : probleme plus petit */  
        return n * fact(n - 1);  
}
```

#### EXEMPLE Trace détaillée de fact(4)

L'appel `fact(4)` déclenche 5 appels empilés sur la pile d'exécution. Phase de descente : `fact(4)` attend `fact(3)`, qui attend `fact(2)`, qui attend `fact(1)`, qui attend `fact(0)`. Phase de remontée :

$$\begin{aligned}\text{fact}(0) &= 1 \\ \text{fact}(1) &= 1 \times \text{fact}(0) = 1 \times 1 = 1 \\ \text{fact}(2) &= 2 \times \text{fact}(1) = 2 \times 1 = 2 \\ \text{fact}(3) &= 3 \times \text{fact}(2) = 3 \times 2 = 6 \\ \text{fact}(4) &= 4 \times \text{fact}(3) = 4 \times 6 = 24\end{aligned}$$

Résultat final :  $4! = 4 \times 3 \times 2 \times 1 \times 1 = 24$ . Chaque appel possède sa propre copie de la variable `n` sur la pile.



Chaîne des appels récursifs de `fact(4)` : flèches pleines  
= descente, flèches pointillées = remontée des résultats.

#### PROPRIÉTÉ Correction de `fact` : démonstration par récurrence

Montrons que `fact(n)` renvoie bien  $n!$  pour tout entier  $n \geq 0$ .

- Initialisation : pour  $n = 0$ , la fonction renvoie 1, et  $0! = 1$  par convention. La propriété est vraie au rang 0.
- Hérédité : supposons que `fact(k)` renvoie  $k!$  pour un entier  $k \geq 0$  quelconque. Alors `fact(k+1)` renvoie  $(k+1) \times \text{fact}(k) = (k+1) \times k! = (k+1)!$ . La propriété est vraie au rang  $k+1$ .
- Conclusion : par récurrence, `fact(n)` renvoie  $n!$  pour tout  $n \geq 0$ .

Ce raisonnement est le modèle général de preuve de toute fonction récursive : vérifier le cas de base, puis supposer le sous-problème correct et en déduire le problème courant.

#### MÉTHODE Concevoir une fonction récursive en 3 questions

Avant d'écrire la moindre ligne de code, réponds par écrit à ces trois questions :

1. Quel est le cas de base ? Le plus petit problème résolvable directement (ex. `fact(0) = 1`).
2. Comment ramener  $n$  à un problème plus petit ? L'appel récursif doit porter sur une donnée strictement réduite (ex. `fact(n-1)`), sinon la récursion ne s'arrêtera jamais.

3. Que combiner au retour? Quelle opération relie le résultat du sous-problème au résultat attendu (ex. multiplier par  $n$ )?

Vérifie enfin sur un petit exemple ( $n = 2$  ou  $3$ ) que la descente atteint bien le cas de base.

**D'autres exemples classiques.** La même méthode s'applique à la somme des  $n$  premiers entiers, à la puissance et à la célèbre suite de Fibonacci (souvent proposée au Bac) :

```
int somme(int n){           /* 1 + 2 + ... + n */
    if (n == 0) return 0; /* cas de base */
    return n + somme(n - 1);
}

double puissance(double x, int n){ /* x^n, n >= 0 */
    if (n == 0) return 1;
    return x * puissance(x, n - 1);
}

int fibo(int n) {           /* suite de Fibonacci */
    if (n <= 1) return n; /* deux cas de base : fibo(0)=0, fibo(1)=1 */
    return fibo(n - 1) + fibo(n - 2);
}
```

#### ATTENTION Récursif ou itératif?

Toute fonction récursive peut s'écrire avec une boucle, et réciproquement. La version récursive est souvent plus lisible et plus proche de la définition mathématique, mais elle consomme de la mémoire (un cadre de pile par appel) et peut être moins efficace : `fibo(n)` récursif recalcule plusieurs fois les mêmes valeurs et devient très lent dès  $n \approx 40$ . En pratique, on choisit la récursivité quand le problème est naturellement récursif (arborescences, fractales), et la boucle quand l'itération est simple. Et surtout : jamais de fonction récursive sans cas de base atteignable — `fact(-1)` avec notre code provoque un débordement de pile, car  $n$  ne vaudra jamais 0. À noter aussi : les factorielles grandissent très vite et dépassent la capacité d'un `long` dès  $n = 21$  ; au-delà, le résultat affiché est faux (dépassement de capacité silencieux).

#### LESAVAIS-TU ? La récursivité est partout

La récursivité est au cœur de nombreux algorithmes avancés : les fractales (figures géométriques auto-similaires comme le flocon de Koch), l'exploration des dossiers d'un disque dur (un dossier contient des dossiers qui contiennent des dossiers...), les jeux d'échecs (le programme évalue un coup en explorant récursivement les réponses possibles de l'adversaire), et même les moteurs de recherche de fichiers de ton système d'exploitation. Les grandes écoles et les entreprises technologiques testent presque toujours la récursivité lors de leurs entretiens d'embauche.

## 6.2 Gestion des erreurs de compilation

**Intuition.** Le compilateur est ton meilleur correcteur : il ne se trompe jamais sur la syntaxe, et ses messages, bien qu'en anglais, contiennent toujours trois informations précieuses : le **fichier**, la **ligne** et la **cause** de l'erreur. Apprendre à les lire, c'est gagner des heures.

Il faut distinguer deux familles d'erreurs :

- les **erreurs syntaxiques** : le programme viole la grammaire du C (point-virgule manquant, parenthèse oubliée...). Le compilateur les détecte et **refuse de produire l'exécutable** ;
- les **erreurs logiques** : le programme compile et s'exécute, mais produit un **résultat faux** (mauvaise formule, boucle infinie, condition inversée). Seuls les tests et le débogueur permettent de les débusquer.

### Build log — Code ::Blocks

```
main.c||In function 'main':
main.c|25|error: expected ';' before 'return'

main.c|31|warning: format '%d' expects 'int *'

main.c|12|error: 'somme' undeclared

=== Build failed: 2 error(s), 1 warning(s) ===
```

fichier | ligne | cause de l'erreur

un warning n'empêche pas la compilation, mais signale souvent un vrai bug (ici le & oublié dans scanf)

Fenêtre de compilation stylisée : chaque message indique le fichier, la ligne et la nature du problème.

| Erreur classique         | Message typique                             | Correction                                             |
|--------------------------|---------------------------------------------|--------------------------------------------------------|
| Point-virgule manquant   | expected ';' before ...                     | Ajouter ; à la fin de l'instruction précédente         |
| Variable non déclarée    | 'x' undeclared                              | Déclarer x avant utilisation (vérifier l'orthographe)  |
| & oublié dans scanf      | warning sur le format, puis plantage        | Écrire <code>scanf("%d", &amp;x);</code>               |
| Accolades déséquilibrées | expected '}' at end of input                | Indenter le code et compter les paires { }             |
| Types incompatibles      | incompatible types ou warning de conversion | Utiliser un cast explicite : <code>(float)a / b</code> |

**ATTENTION** Corrige toujours la **PREMIÈRE** erreur d'abord

Une erreur signalée à la ligne 25 est très souvent causée par un oubli à la ligne 24 (par

exemple un point-virgule manquant). Une seule faute peut déclencher une cascade de 20 messages en aval. La bonne démarche : corriger la première erreur de la liste, recompiler, et constater que la moitié des messages a disparu. Ne t'acharne jamais sur la dernière erreur avant d'avoir traité la première.

**Le débogueur de l'IDE.** Pour traquer les erreurs logiques, Code ::Blocks et Dev-C++ intègrent un débogueur. La démarche :

1. placer un **point d'arrêt** (breakpoint) sur la ligne suspecte (clic dans la marge ou touche F5);
2. lancer le programme en mode Debug : l'exécution se fige au point d'arrêt;
3. avancer **pas à pas** (F7 : entrer dans les fonctions; F8 : ligne suivante);
4. observer dans la fenêtre Watches la **valeur des variables** à chaque étape et repérer le moment où elle devient fausse.

C'est l'outil idéal pour visualiser concrètement la pile d'appels d'une fonction récursive : tu verras `fact` apparaître plusieurs fois avec des valeurs de `n` différentes.

### 6.3 Projet guidé : deux applications complètes

#### EXPÉRIENCE Projet 1 : calculatrice à menu

**Objectif :** produire un exécutable complet combinant boucle `do...while`, `switch` et fonctions.

**Matériel :** un ordinateur avec Code ::Blocks (ou Dev-C++) installé.

**Étapes :**

1. Créer un projet console C dans Code ::Blocks, écrire le code ci-dessous.
2. Compiler (F9), corriger les éventuelles erreurs en commençant par la première.
3. Tester chaque opération, y compris la division par zéro, puis quitter avec le choix 0.

```
#include <stdio.h>

double addition(double a, double b) { return a + b; }
double produit(double a, double b) { return a * b; }

int main(void) {
    int choix;
    double x, y;
    do {
        printf("\n=== CALCULATRICE ===\n");
        printf("1. Addition\n2. Multiplication\n0. Quitter\n");
        printf("Votre choix : ");
        scanf("%d", &choix);
        switch (choix) {
            case 1:
```

```

        printf("Deux nombres : ");
        scanf("%lf %lf", &x, &y);
        printf("Resultat : %.2f\n", addition(x, y));
        break;
    case 2:
        printf("Deux nombres : ");
        scanf("%lf %lf", &x, &y);
        printf("Resultat : %.2f\n", produit(x, y));
        break;
    case 0:
        printf("Au revoir !\n");
        break;
    default:
        printf("Choix invalide.\n");
    }
} while (choix != 0); /* la boucle recommence tant qu'on ne quitte pas */
return 0;
}

```

**Observation :** le menu réapparaît après chaque opération grâce au `do...while`, qui garantit au moins un affichage. Le `switch` aiguille vers la bonne fonction. Remarque le format `%lf` dans `scanf` : il est obligatoire pour lire un double (alors que `printf` accepte `%f` pour un double).

#### EXERCICE RÉSOLU 6 Projet 2 : mini-gestionnaire de tontine

**Énoncé.** Une tontine regroupe au plus 50 membres. Écrire un programme C qui : (a) saisit les membres (nom, mise mensuelle) dans un tableau d'enregistrements ; (b) trie les membres par mise décroissante (tri par sélection) ; (c) recherche un membre par son nom (recherche séquentielle) ; (d) affiche le total collecté chaque mois.

#### SOLUTION

**Solution détaillée.** On déclare une `struct Membre`, un tableau `t` et un compteur `n`. Le tri par sélection échange des enregistrements complets ; la recherche compare les chaînes avec `strcmp` (bibliothèque `string.h`).

```

#include <stdio.h>
#include <string.h>

#define MAX 50

typedef struct {
    char nom[30];
    float mise; /* mise mensuelle en FCFA */
} Membre;

/* Tri par selection : mises décroissantes */
void trier(Membre t[], int n) {
    int i, j, max;
    Membre tmp;

```

```

    for (i = 0; i < n - 1; i++){
        max=i;
        for (j = i + 1; j < n; j++)
            if (t[j].mise > t[max].mise)
                max=j;
        tmp = t[i]; t[i] = t[max]; t[max] = tmp;
    }
}

/* Recherche sequentielle par nom : retourne l'indice ou -1 */
int rechercher(Membre t[], int n, char nom[]) {
    int i;
    for (i = 0; i < n; i++)
        if (strcmp(t[i].nom, nom) == 0)
            return i;
    return -1;
}

int main(void){
    Membre t[MAX];
    int n, i, pos;
    float total = 0;
    char nomCherche[30];

    printf("Nombre de membres (max %d) : ", MAX);
    scanf("%d", &n);
    for (i = 0; i < n; i++){
        printf("Nom du membre %d : ", i+1);
        scanf("%s", t[i].nom);
        printf("Mise mensuelle (FCFA) : ");
        scanf("%f", &t[i].mise);
    }

    trier(t, n);
    printf("\n--- Membres tries par mise decroissante ---\n");
    for (i = 0; i < n; i++){
        printf("%-15s %10.0f FCFA\n", t[i].nom, t[i].mise);
        total += t[i].mise;
    }
    printf("Total mensuel collecte : %.0f FCFA\n", total);

    printf("\nNom a rechercher : ");
    scanf("%s", nomCherche);
    pos = rechercher(t, n, nomCherche);
    if (pos >= 0)
        printf("%s mise %.0f FCFA (rang %d).\n",
            t[pos].nom, t[pos].mise, pos + 1);
    else
        printf("Membre introuvable.\n");
    return 0;
}

```

Remarque la transmission du tableau `t` aux fonctions : en C, un tableau est passé par adresse, donc `trier` modifie directement le tableau du `main`. Le `typedef` permet d'écrire simplement `Membre` au lieu de `struct Membre`. Enfin, `strcmp(a, b)` renvoie 0 quand les deux chaînes sont identiques : on teste donc `== 0`, et non `== 1`.

**À RETENIR** L'essentiel de la section

- Une fonction récursive s'appelle elle-même : cas de base (arrêt) + cas récursif (problème réduit). Sans cas de base atteignable : débordement de pile.
- Conception en 3 questions : cas de base ? réduction du problème ? combinaison au retour ?
- Le compilateur indique fichier, ligne et cause : corriger la première erreur d'abord, puis recompiler.
- Erreur syntaxique ≠ erreur logique : la seconde se traque au débogueur (points d'arrêt, pas à pas, observation des variables).
- Un programme complet combine structures de contrôle, fonctions, tableaux et enregistrements : tu sais désormais en produire un de bout en bout.

**EXERCICE 6** À toi de jouer

1. Écris une fonction récursive `int sommeChiffres(int n)` qui retourne la somme des chiffres de  $n$  (ex.  $4572 \rightarrow 18$ ). Indice : cas de base  $n = 0$ , sinon  $n \bmod 10 + \text{sommeChiffres}(n/10)$ .
2. Le programme suivant ne compile pas ; trouve et corrige les trois erreurs :

```
#include <stdio.h>
int main(void) {
    int age;
    printf("Age : ");
    scanf("%d", age)
    if (age >= 18)
        printf("Majeur\n");
    return 0;
```

3. Ajoute au gestionnaire de tontine une option « modifier la mise d'un membre » en réutilisant la fonction `rechercher`.

**CORRIGÉ** Exercice n°2

Trois erreurs : (1) il manque le `&` devant `age` dans `scanf` : `scanf("%d", &age);` ; (2) il manque le point-virgule après cette instruction ; (3) il manque l'accolade fermante `}` du `main` avant la fin du fichier. Le compilateur signalera d'abord l'erreur de point-virgule à la ligne du `if` (ligne suivante !), ce qui illustre la règle : la ligne signalée n'est pas toujours la ligne fautive.

## 7 ACTIVITÉ D'INTÉGRATION

### 7.1 Objectifs de l'activité

À l'issue de cette activité d'intégration, tu seras capable de :

- analyser un besoin réel de gestion et le traduire en un cahier des charges de programme ;
- déclarer et manipuler un **enregistrement** (struct) et un tableau d'enregistrements ;
- structurer un programme complet en **fonctions** avec transmission de paramètres ;
- utiliser les **structures alternatives** (switch), les **boucles** (do...while, for) et les **entrées-sorties formatées** (printf, scanf) ;
- implémenter la **recherche séquentielle** et le **tri par sélection** sur un tableau de structures ;
- compiler un programme, lire et corriger les erreurs de compilation, puis exécuter **pas à pas** au débogueur pour vérifier le comportement d'un algorithme ;
- produire un **exécutable complet** et le présenter lors d'une courte soutenance.

### 7.2 Situation

Mme Etoundi, commerçante au marché Mokolo à Yaoundé, est la trésorière de la tontine « Étoile du Centre », une coopérative de quartier. Chaque mois, les membres cotisent un montant variable (entre 5000 et 100000 FCFA). Jusqu'ici, elle note tout dans un cahier : nom du membre, montant cotisé, mois de la cotisation. Problème : le cahier s'abîme, les recherches sont lentes, les totaux sont souvent faux, et les membres réclament un classement des plus gros cotisants pour l'attribution des tours de ramassage.

Sachant que tu suis le cours de TI au lycée, elle te sollicite, toi et ton binôme, pour développer un petit programme en langage C, exécuté sur l'ordinateur du cybercafé voisin, qui gère sa tontine. Elle te fournit un jeu d'essai réel de 5 membres :

| N° | Nom du membre  | Montant (FCFA) | Mois    |
|----|----------------|----------------|---------|
| 1  | Ngo Bell       | 25000          | Janvier |
| 2  | Kamdem Paul    | 50000          | Janvier |
| 3  | Fouda Marie    | 10000          | Février |
| 4  | Mbarga Jean    | 75000          | Janvier |
| 5  | Aissatou Haman | 30000          | Février |

Le programme attendu doit afficher un **menu principal** proposant : (1) enregistrer un nouveau membre avec saisie contrôlée du montant (obligatoirement entre 5000 et 100000 FCFA) ; (2) afficher la liste formatée des membres ; (3) rechercher un membre par son nom ; (4) trier les membres par montant décroissant ; (5) afficher le total des cotisations ; (6) quitter. Le total attendu pour le jeu d'essai est de 190000 FCFA, et après tri, Mbarga Jean doit apparaître en tête.

### 7.3 Consignes

Travail en binôme, sur Code ::Blocks ou Dev-C++. Durée : 2 heures, suivies d'une soutenance de 3 minutes par binôme.

1. **Modélisation.** Déclare une structure `Membre` contenant les champs `nom` (chaîne de 50 caractères), `montant` (entier) et `mois` (chaîne de 15 caractères). Déclare une constante `MAX` valant 100 (capacité maximale de la tontine) et un tableau global de `Membre`.
2. **Saisie contrôlée.** Écris la fonction `enregistrer` qui ajoute un membre. Le montant doit être redemandé tant qu'il n'est pas compris entre 5000 et 100000 FCFA (utilise une boucle `do...while`).
3. **Affichage formaté.** Écris la fonction `afficher` qui présente la liste sous forme de tableau aligné grâce aux formats de `printf` (`%-20s, %10d`, etc.).
4. **Recherche.** Écris la fonction `rechercher` qui applique la recherche séquentielle sur le champ `nom` (comparaison avec `strcmp`) et affiche le membre trouvé ou un message d'échec.
5. **Tri.** Écris la fonction `trier` qui trie le tableau par montant décroissant selon le tri par sélection, en échangeant des structures entières.
6. **Total.** Écris la fonction `totalCotisations` qui reçoit le tableau et le nombre de membres, puis retourne la somme des montants.
7. **Menu.** Dans `main`, construis le menu avec une boucle `do...while` et un `switch` à 6 options. Pré-charge les 5 membres du jeu d'essai pour faciliter les tests.
8. **Compilation et débogage.** Compile, lis chaque message d'erreur, corrige. Exécute ensuite le tri **pas à pas** au débogueur (point d'arrêt dans `trier`) et vérifie chaque échange sur le jeu d'essai. Vérifie enfin que le total affiché vaut bien 190000 FCFA.
9. **Soutenance.** En 3 minutes : démonstration du programme, explication d'une fonction au choix, et une difficulté rencontrée avec sa solution.

### 7.4 Ressources à mobiliser

#### À RETENIR Boîte à outils de la leçon

- **Enregistrement** : `struct Membre { char nom[50]; int montant; char mois[15]; };` — on accède aux champs par le point : `m.montant`. Attention au point-virgule final après l'accolade fermante.
- **Entrées-sorties** : `printf("%d", x)`, `scanf("%d", &x)`; pour une chaîne : `scanf("%s", m.nom)` (pas de `&` devant un tableau, car un tableau est déjà une adresse). Comparaison de chaînes : `strcmp(a, b) == 0` (bibliothèque `string.h`).
- **Menu** : boucle `do { ... } while (choix != 6);` avec `switch (choix)` et un `case` par option, sans oublier les `break`.

- Recherche séquentielle : parcourir du premier au dernier élément; si `strcmp(t[i].nom, cherche) == 0`, on a trouvé à l'indice *i* et on peut sortir avec `break`.
- Tri par sélection décroissant : pour *i* de 0 à *n* - 2, chercher l'indice *iMax* du maximum dans `t[i..n-1]`, puis échanger `t[i]` et `t[iMax]` via une variable temporaire de type `struct Membre`.
- Transmission de paramètres : un tableau passé à une fonction est transmis par adresse : les modifications faites dans la fonction sont visibles dans `main`. Une valeur de retour (`long`) s'obtient avec `return`.

## 7.5 Démarche et corrigé commenté

### EXERCICE RÉSOLU 7 Le gestionnaire de tontine — solution complète

Étape 1 — Analyse. Les données sont hétérogènes (chaînes + entier) : c'est le cas typique d'un `struct`. Le nombre de membres varie : on gère un tableau `tontine` et un compteur global `nb`. Chaque fonctionnalité du menu devient une fonction, ce qui rend le programme lisible et testable séparément.

Étape 2 — Programme complet.

```
#include <stdio.h>
#include <string.h>

#define MAX 100

/* Declaration de l'enregistrement */
struct Membre {
    char nom[50];
    int montant;
    char mois[15];
};

/* Variables globales : tableau et effectif */
struct Membre tontine[MAX];
int nb = 0;

/* 1. Enregistrement avec saisie controlée */
void enregistrer() {
    if (nb >= MAX) {
        printf("Tontine pleine !\n");
        return;
    }
    printf("Nom du membre (sans espace) : ");
    scanf("%s", tontine[nb].nom);
    printf("Mois de cotisation : ");
    scanf("%s", tontine[nb].mois);
    /* Saisie controlée : on recommence tant que le montant est invalide */
    do {
```

```

        printf("Montant (5000 a 100000 FCFA) : ");
        scanf("%d", &tontine[nb].montant);
        if (tontine[nb].montant < 5000 || tontine[nb].montant > 100000)
            printf("Montant invalide, recommencez.\n");
    } while (tontine[nb].montant < 5000 || tontine[nb].montant > 100000);
    nb++;
    printf("Membre enregistre avec succes.\n");
}

/* 2. Affichage formate de la liste */
void afficher() {
    int i;
    printf("\n%-20s %-15s %10s\n", "NOM", "MOIS", "MONTANT");
    printf("-----\n");
    for (i = 0; i < nb; i++)
        printf("%-20s %-15s %10d\n",
            tontine[i].nom, tontine[i].mois, tontine[i].montant);
}

/* 3. Recherche sequentielle par nom */
void rechercher() {
    char cherche[50];
    int i, trouve = 0;
    printf("Nom a rechercher : ");
    scanf("%s", cherche);
    for (i = 0; i < nb; i++) {
        if (strcmp(tontine[i].nom, cherche) == 0) {
            printf("Trouve : %s a cotise %d FCFA en %s.\n",
                tontine[i].nom, tontine[i].montant, tontine[i].mois);
            trouve = 1;
            break; /* on arrete des la premiere occurrence */
        }
    }
    if (!trouve)
        printf("Aucun membre de ce nom.\n");
}

/* 4. Tri par selection : montants decroissants */
void trier() {
    int i, j, iMax;
    struct Membre temp;
    for (i = 0; i < nb - 1; i++) {
        iMax = i;
        for (j = i + 1; j < nb; j++)
            if (tontine[j].montant > tontine[iMax].montant)
                iMax = j;
        /* Echange des structures entieres */
        temp = tontine[i];
        tontine[i] = tontine[iMax];
        tontine[iMax] = temp;
    }
    printf("Liste trie par montant decroissant.\n");
}

/* 5. Total des cotisations (tableau + taille en parametres) */
long totalCotisations(struct Membre t[], int n) {
    long s = 0;

```

```

int i;
for (i = 0; i < n; i++)
    s += t[i].montant;
return s;
}

int main() {
    int choix;
    /* Pre-chargement du jeu d'essai de Mme Etoundi */
    strcpy(tontine[0].nom, "Ngo"); tontine[0].montant = 25000; strcpy(tontine[0].mois, "Janvier");
    strcpy(tontine[1].nom, "Kamdem"); tontine[1].montant = 50000; strcpy(tontine[1].mois, "Janvier");
    strcpy(tontine[2].nom, "Fouda"); tontine[2].montant = 10000; strcpy(tontine[2].mois, "Fevrier");
    strcpy(tontine[3].nom, "Mbarga"); tontine[3].montant = 75000; strcpy(tontine[3].mois, "Janvier");
    strcpy(tontine[4].nom, "Aissatou"); tontine[4].montant = 30000; strcpy(tontine[4].mois, "Fevrier");
    nb = 5;

    do {
        printf("\n===== TONTINE ETOILE DU CENTRE =====\n");
        printf("1. Enregistrer un membre\n");
        printf("2. Afficher la liste\n");
        printf("3. Rechercher un membre\n");
        printf("4. Trier par montant decroissant\n");
        printf("5. Total des cotisations\n");
        printf("6. Quitter\n");
        printf("Votre choix : ");
        scanf("%d", &choix);
        switch (choix) {
            case 1: enregistrer(); break;
            case 2: afficher(); break;
            case 3: rechercher(); break;
            case 4: trier(); break;
            case 5: printf("Total : %ld FCFA\n",
                          totalCotisations(tontine, nb)); break;
            case 6: printf("Au revoir, Mme Etoundi !\n"); break;
            default: printf("Choix invalide.\n");
        }
    } while (choix != 6);
    return 0;
}

```

### Étape 3 — Justifications.

- Le total est déclaré `long` car une tontine de 100 membres à 100000 FCFA atteint 10000000 FCFA : valeur encore compatible avec `int`, mais `long` est plus sûr (format `%ld` obligatoire dans `printf`).
- On n'écrit pas `&` devant `tontine[nb].nom` dans `scanf("%s", ...)` : un tableau de caractères est déjà une adresse. En revanche, le `&` est indispensable pour l'entier : `scanf("%d", &tontine[nb].montant)`.
- `strcmp` retourne 0 en cas d'égalité : le test est `== 0`, jamais `!= 0`.
- Les chaînes du pré-chargement sont copiées avec `strcpy` : on ne peut pas affecter une chaîne à un tableau par = après sa déclaration.
- `scanf("%s", ...)` s'arrête au premier espace : c'est pourquoi le jeu d'essai utilise les

seuls noms de famille (Ngo, Kamdem...). Saisir « Ngo Bell » couperait la chaîne et décalerait la saisie suivante. Une version professionnelle utiliserait `fgets`, hors programme de Terminale.

Étape 4 — Vérification au débogueur. Point d'arrêt sur la ligne de l'échange dans `trier`. Premier passage ( $i = 0$ ) : le maximum du tableau est 75000 (Mbarga, indice 3); après échange, Mbarga occupe la position 0. Deuxième passage ( $i = 1$ ) : le maximum de la sous-partie restante est 50000 (Kamdem). Après  $n - 1 = 4$  passages, l'ordre obtenu est : Mbarga (75000), Kamdem (50000), Aissatou (30000), Ngo (25000), Fouda (10000). L'option 5 affiche bien `Total : 190000 FCFA` : le cahier des charges est respecté.

#### MÉTHODE Vérifier un tri par sélection avec une trace d'exécution

Pour valider ton tri sur papier (compétence exigée au Bac) avant même de compiler :

1. Écris le tableau initial sur une ligne, un montant par case, avec les indices 0 à  $n - 1$ .
2. Pour chaque valeur de  $i$  de 0 à  $n - 2$ , souligne la sous-partie  $t[i..n - 1]$  et entoure son maximum : c'est  $i_{\text{Max}}$ .
3. Barre les cases  $i$  et  $i_{\text{Max}}$  et écris les valeurs échangées sur la ligne suivante.
4. Vérifie qu'à la fin du passage  $i$ , les  $i + 1$  premières cases sont définitivement triées.
5. Compte les passages : il doit y en avoir exactement  $n - 1$ , soit 4 pour 5 membres.

Cette trace est aussi ce que tu observes, case par case, dans la fenêtre Watch du débogueur de Code ::Blocks.

#### ATTENTION Erreurs fréquentes rencontrées pendant l'activité

- Point-virgule oublié après l'accolade fermante de la déclaration du `struct` : le compilateur signale une erreur à la ligne suivante, ce qui déroute souvent.
- `break` oublié dans un `case` : les options du menu s'enchaînent toutes (exécution en cascade).
- Échange sans variable temporaire dans le tri : la première valeur écrasée est perdue définitivement.
- Condition de boucle inversée dans la saisie contrôlée : `tester < 5000 || > 100000` (montant invalide), et non `&&` — un montant ne peut pas être à la fois inférieur à 5000 et supérieur à 100000, la boucle ne recommencerait jamais.
- Comparaison de chaînes avec `==` : cela compare des adresses, pas les contenus ; toujours utiliser `strcmp`.
- Dépassement de tableau : saisir un nom de plus de 49 caractères avec `scanf("%s",`

...) écrase la mémoire voisine (comportement imprévisible). En production, on limite avec %49s.

- Format inadapté : afficher un `long` avec `%d` au lieu de `%ld` produit une valeur fausse ou un avertissement du compilateur.

#### LE SAVAIS-TU ? Les tontines, ancêtres camerounais de la microfinance numérique

La tontine est une institution d'épargne collective très répandue au Cameroun. Aujourd'hui, des startups de la fintech camerounaise la digitalisent : les cotisations passent par MTN Mobile Money ou Orange Money, et le classement des bénéficiaires est calculé par des programmes qui utilisent exactement les algorithmes que tu viens d'écrire — recherche dans un tableau, tri par sélection ou algorithmes plus rapides, accumulation de totaux. Ton programme de 150 lignes est le cœur simplifié de ces applications professionnelles.

## 7.6 Bilan de l'activité

Cette activité a permis de mobiliser simultanément l'ensemble des savoir-faire de la leçon dans un cadre authentique. Tu as d'abord constaté qu'un besoin réel (la tontine de Mme Etoundi) se traduit naturellement par des choix de conception précis : données hétérogènes → `struct` ; collection → tableau ; traitements distincts → fonctions ; interaction répétée → boucle `do...while` et `switch`. Tu as ensuite vérifié expérimentalement, au débogueur, que le tri par sélection effectue bien  $n - 1$  passages et que chaque échange place le maximum de la sous-partie restante en tête. Enfin, la lecture des messages du compilateur t'a montré qu'une erreur de syntaxe se diagnostique méthodiquement, ligne par ligne. Le programme produit est un véritable exécutable professionnel en miniature : c'est exactement la démarche suivie par les développeurs camerounais qui conçoivent les applications de gestion des coopératives, des écoles ou des microfinances.

## 8 MÉTHODES ET EXERCICES RÉSOLUS

## 9 PREMIERS PROGRAMMES EN C : STRUCTURE, TYPES, ENTRÉES-SORTIES

### PROPRIÉTÉ Types de base, constantes et entrées-sorties formatées

En C, tout programme manipule des données typées. Les types fondamentaux sont :

- `int` : entier signé (généralement 32 bits, intervalle  $\approx \pm 2 \times 10^9$ ).
- `float` : réel simple précision (6–7 chiffres significatifs).
- `double` : réel double précision (15–16 chiffres) ; par défaut pour les littéraux à virgule.
- `char` : caractère unique (codé sur 1 octet, valeur ASCII).

Une **constante** nommée se déclare avec `const` (ex. `const int MAX = 100;`) : elle occupe de la mémoire mais est en lecture seule. Les bibliothèques standard `stdio.h` (entrées-sorties) et `math.h` (fonctions mathématiques : `sqrt`, `cos`, `fabs`, `pow`...) s'incluent par `#include <...>`.

### 9.1 Méthode 1 : Écrire et exécuter pas à pas un programme C avec un IDE

#### MÉTHODE De l'algorithme au programme exécutable sous Code ::Blocks

Pour écrire, compiler et exécuter pas à pas un programme C à l'aide d'un IDE (Code ::Blocks, Dev-C++), suis toujours la même démarche :

1. Analyser le problème : identifier les données d'entrée, les traitements et les résultats attendus. Écrire l'algorithme en pseudo-code d'abord.
2. Créer le projet : dans Code ::Blocks, `File` → `New` → `Project` → `Console application` → `C`. Le fichier principal s'appelle `main.c`.
3. Structurer le programme : en-têtes de bibliothèques (`#include <stdio.h>`, `#include <math.h>`), déclarations des constantes et variables, puis la fonction `main()` qui retourne 0.
4. Compiler : touche F9 (Build and Run). Lire attentivement la fenêtre Build log : chaque erreur indique le fichier, la ligne et la nature du problème (erreur de syntaxe, variable non déclarée, type incompatible, linker error pour `math.h` sans `-lm`...).

5. Exécuter pas à pas : placer un point d'arrêt (clic dans la marge ou F5), lancer le débogueur (F8), avancer instruction par instruction (F7 = Next line / pas à pas principal) en observant les valeurs des variables dans la fenêtre Watches.
6. Tester : prévoir un jeu d'essai couvrant les cas normaux et les cas limites (valeur nulle, valeur négative, borne supérieure, saisie invalide).

Réflexes à retenir : chaque instruction se termine par ; ; les formats de `printf/scanf` sont `%d` (int), `%f` (float), `%lf` (double dans `scanf`), `%c` (char) ; dans `scanf`, on passe l'adresse de la variable avec `&` (sauf pour un tableau de caractères, déjà adresse).

#### EXERCICE RÉSOLU 8 Programme de facturation d'un cybercafé

Énoncé (type Bac). Un cybercafé de Yaoundé facture la connexion Internet à 250 FCFA l'heure. Une remise de 10 % est accordée pour toute durée strictement supérieure à 5 heures. Écrire un programme C qui saisit le nombre d'heures (entier positif), calcule et affiche le montant à payer.

#### SOLUTION

Solution détaillée.

Étape 1 — Analyse. Entrée : `heures` (int). Traitement : `montant brut = 250 × heures` ; si `heures > 5`, appliquer une remise de 10 %. Sortie : le montant à payer (float).

Étape 2 — Programme.

```
#include <stdio.h>

int main(void)
{
    const int PRIX_HEURE = 250; // tarif horaire en FCFA
    int heures;
    float montant;

    printf("Nombre d'heures de connexion : ");
    scanf("%d", &heures); // & obligatoire : adresse de la variable

    montant = PRIX_HEURE * heures; // conversion implicite int -> float
    if (heures > 5) {
        montant = montant * 0.9f; // remise de 10 % (littéral float 0.9f)
    }

    printf("Montant à payer : %.2f FCFA\\", montant);
    return 0;
}
```

Étape 3 — Vérification par jeu d'essai.

- `heures = 3` : `montant = 250 × 3 = 750 FCFA` (pas de remise, car  $3 \leq 5$ ). Affichage : 750.00 FCFA.

- heures = 8 : brut =  $250 \times 8 = 2000$  ; remise :  $2000 \times 0,9 = 1800$  FCFA. Affichage : 1800.00 FCFA.
- Cas limite heures = 5 : la condition  $5 > 5$  est fausse, donc pas de remise : 1250 FCFA. C'est bien ce que demande l'énoncé (« strictement supérieure »).

Étape 4 — Exécution pas à pas. On place un point d'arrêt sur la ligne du `scanf`, on lance le débogueur (F8) et on avance avec F7 : dans la fenêtre Watches, on observe `heures` qui passe à 8 après la saisie, puis `montant` qui vaut 2000 puis 1800 après le `if`.

Conclusion. Le programme compile sans erreur et produit les résultats attendus sur tout le jeu d'essai : il est correct.

## 10 OPÉRATEURS, CONVERSIONS DE TYPES ET EXPRESSIONS

### PROPRIÉTÉ Opérateurs, conversions et priorités en C

Opérateurs arithmétiques : `+` `-` `*` `/` `%` (modulo, reste de la division entière). Incrément/décément : `++x` (préfixe, incrémente puis utilise), `x++` (postfixe, utilise puis incrémente); idem pour `--`. Opérateurs d'affectation composée : `+=` `-=` `*=` `/=` `%=` (ex. `x += 3`  $\equiv$  `x = x + 3`). Opérateurs relationnels : `<` `<=` `>` `>=` `==` `!=` (renvoient 1 si vrai, 0 si faux). Opérateurs logiques : `&&` (ET), `||` (OU), `!` (NON) — évaluation paresseuse (short-circuit). Conversion implicite (promotion) : dans une expression mixte, le type « plus petit » est promu vers le « plus grand » (`int`  $\rightarrow$  `float`  $\rightarrow$  `double`). Conversion explicite (cast) : `(type)expression` (ex. `(float)somme / n` pour forcer une division réelle). Tableau de priorité (du plus fort au plus faible) :

1. `() [] -> . ++ --` (postfixes)
2. `++ -- + - ! (type) * & sizeof` (unaires, préfixes)
3. `*` `/` `%`
4. `+` `-`
5. `<` `<=` `>` `>=`
6. `==` `!=`
7. `&&`
8. `||`
9. `?:` (conditionnel)
10. `= += -= *= /= %=` (associativité à droite)

11. ,

En cas de doute, parenthéser.

## 11 STRUCTURES ALTERNATIVES ET BOUCLES

### 11.1 Méthode 2 : Choisir et rédiger la bonne structure de contrôle

#### MÉTHODE Structures alternatives et répétitives

1. Alternative simple ou double : `if (condition) { ... } else { ... }`. La condition est toute expression scalaire (0 = faux, non-zéro = vrai).
2. Alternative multiple (cas disjoints sur une même variable entière/caractère) : `switch (variable) { case val1: ... break; case val2: ... break; default: ... }`. Ne jamais oublier `break` (sauf fall-through volontaire documenté).
3. Boucle `for` : nombre d'itérations connu à l'avance. Syntaxe : `for (init; cond; incr) { ... }`. L'initialisation, la condition et l'incrément sont optionnelles (mais les ; restent).
4. Boucle `while` : répétition conditionnelle pouvant ne jamais s'exécuter (test en entrée). `while (cond) { ... }`.
5. Boucle `do...while` : répétition s'exécutant au moins une fois (saisie contrôlée, menu). `do { ... } while (cond);` — attention au ; final.
6. Garantir la terminaison : au moins une variable de la condition doit être modifiée dans le corps de la boucle.
7. Accumulateurs et compteurs : initialiser avant la boucle (`somme = 0;`, `compteur = 0;`), mettre à jour dans la boucle.

#### EXERCICE RÉSOLU 9 Statistiques des notes d'une classe de Terminale TI

Énoncé (type Bac). Écrire un programme C qui saisit les notes d'informatique des élèves d'une classe (la saisie s'arrête quand on entre une note négative), puis affiche le nombre de notes saisies, la moyenne de la classe et le nombre d'élèves admis (note  $\geq 10$ ). Chaque note saisie doit être validée : si elle dépasse 20, elle est rejetée et redemandée.

**SOLUTION**

Solution détaillée.

Étape 1 — Choix des structures. Le nombre de notes est inconnu → boucle `while` pilotée par la valeur saisie (sentinelle négative). La validation d'une note doit s'exécuter au moins une fois → `do...while`. Le test d'admission → `if`.

Étape 2 — Programme.

```
#include <stdio.h>

int main(void)
{
    float note, somme = 0.0f;
    int effectif = 0, admis = 0;

    printf("Entrez une note (note negative pour terminer) : ");
    scanf("%f", &note);

    while (note >= 0) {
        do { // validation : la note doit etre <= 20
            if (note > 20) {
                printf("Note invalide, ressaisissez : ");
                scanf("%f", &note);
            }
        } while (note > 20);

        if (note >= 0) { // la ressaisie a pu donner une sentinelle
            somme = somme + note;
            effectif++;
            if (note >= 10) {
                admis++;
            }
            printf("Entrez une note (note negative pour terminer) : ");
            scanf("%f", &note);
        }
    }

    if (effectif > 0) {
        printf("Effectif : %d\\ ", effectif);
        printf("Moyenne : %.2f / 20\\ ", somme / effectif);
        printf("Admis : %d\\ ", admis);
    } else {
        printf("Aucune note saisie.\\ ");
    }
    return 0;
}
```

Étape 3 — Trace d'exécution avec les saisies 12, 7, 25 → 15, -1 :

```
note = 12 :  somme = 12, effectif = 1, admis = 1
note = 7 :   somme = 19, effectif = 2, admis = 1
note = 25 :  rejetée (do...while), ressaisie 15 :
              somme = 34, effectif = 3, admis = 2
note = -1 :  sortie de boucle
```

Moyenne =  $\frac{34}{3} \approx 11,33$ . Affichage : Effectif : 3, Moyenne : 11.33 / 20, Admis : 2.

Étape 4 — Justifications. La boucle `while` termine car la variable `note` est re-lue à chaque itération ; la division `somme / effectif` est protégée par le test `effectif > 0` (sinon division par zéro) ; `somme` étant un `float`, la division est réelle et non entière.

Conclusion. Le programme affiche correctement l'effectif (3), la moyenne (11,33/20) et le nombre d'admis (2), en rejetant les notes invalides.

## 12 FONCTIONS, PORTÉE DES VARIABLES ET ENREGISTREMENTS

### 12.1 Méthode 3 : Déclarer et appeler des fonctions, gérer la portée des variables

#### MÉTHODE Concevoir une fonction en C

1. Identifier le sous-problème : une tâche répétée ou autonome (calcul, test, affichage) mérite une fonction.
2. Définir l'interface : le type de retour (ou `void` si la fonction ne renvoie rien), le nom, et la liste des paramètres typés. Exemple : `float moyenne(float a, float b);`.
3. Placer le prototype avant `main` (ou la définition complète), puis la définition après `main`.
4. Appeler la fonction avec des arguments du bon type : `m = moyenne(n1, n2);`.
5. Respecter la portée : une variable déclarée dans une fonction est **locale** (invisible ailleurs, détruite à la fin de l'appel) ; une variable déclarée hors de toute fonction est **globale** (visible partout — à utiliser avec parcimonie).
6. Transmission des paramètres : en C, le passage se fait par valeur : la fonction travaille sur une copie. Pour modifier la variable de l'appelant, on passe son adresse (pointeur) : `void echanger(int *a, int *b)` appelé par `echanger(&x, &y);`.

Réflexe Bac : un même nom peut désigner une variable globale et une variable locale ;

c'est la locale qui masque la globale dans la fonction.

#### EXERCICE RÉSOLU 10 Fonctions et portée : conversion et équation du second degré

Énoncé (type Bac). 1) Écrire une fonction `float discriminant(float a, float b, float c)` qui retourne  $b^2 - 4ac$ , en utilisant `math.h`. 2) Écrire un programme complet qui saisit  $a, b, c$  ( $a \neq 0$ ) et affiche les solutions réelles de  $ax^2 + bx + c = 0$  selon le signe du discriminant. 3) On considère le code suivant : que vaut  $x$  après l'appel `modifier(x)` ? Justifier.

```
void modifier(int n) { n = n + 10; }
int x = 5;
```

#### SOLUTION

Solution détaillée.

1) et 2) — Programme complet.

```
#include <stdio.h>
#include <math.h> // pour sqrt ; compiler avec -lm sous gcc (ex. gcc main.c -o main -lm)

float discriminant(float a, float b, float c)
{
    return b * b - 4 * a * c; // variables locales a, b, c
}

int main(void)
{
    float a, b, c, delta;

    printf("Coefficients a b c : ");
    scanf("%f %f %f", &a, &b, &c);

    delta = discriminant(a, b, c); // appel : passage par valeur

    if (delta < 0) {
        printf("Pas de solution reelle.\n");
    } else if (delta == 0) {
        printf("Solution double : x = %.2f\n", -b / (2 * a));
    } else {
        printf("x1 = %.2f\n", (-b - sqrt(delta)) / (2 * a));
        printf("x2 = %.2f\n", (-b + sqrt(delta)) / (2 * a));
    }
    return 0;
}
```

Vérification avec  $a = 1, b = -3, c = 2$  :  $\Delta = (-3)^2 - 4 \times 1 \times 2 = 9 - 8 = 1 > 0$ , d'où  $x_1 = \frac{3-1}{2} = 1$  et  $x_2 = \frac{3+1}{2} = 2$ . Le programme affiche  $x_1 = 1.00$  et  $x_2 = 2.00$  : correct.

3) — Question de portée et de transmission. Après `modifier(x)`, la variable globale  $x$  vaut toujours 5. Justification : le paramètre  $n$  est une variable locale qui reçoit une copie de la

valeur de  $x$  (passage par valeur); l'affectation  $n = n + 10$  modifie uniquement cette copie, détruite à la fin de l'appel. Pour modifier réellement  $x$ , il faudrait passer son adresse :

```
void modifier(int *n) { *n = *n + 10; }  
/* appel : */ modifier(&x); // x vaut alors 15
```

Conclusion. Le programme résout correctement l'équation du second degré grâce à la fonction `discriminant`, et la question 3 illustre la règle fondamentale : en C, tout paramètre est transmis par valeur; seul le passage d'adresse permet de modifier la variable de l'appelant.

## 12.2 Méthode 4 : Intégrer un enregistrement (struct) dans un programme

### MÉTHODE Déclarer et exploiter une structure

1. Modéliser l'objet : lister les champs (attributs) et choisir le type de chacun. Exemple : un élève = matricule (int), nom (chaîne), moyenne (float).
2. Déclarer le type avant `main` :

```
struct Eleve {  
    int matricule;  
    char nom[30];  
    float moyenne;  
};
```

Ne pas oublier le ; après l'accolade fermante.

3. Déclarer les variables : `struct Eleve e;` ou un tableau `struct Eleve classe[50];`.
4. Accéder aux champs avec l'opérateur point : `e.moyenne = 14.5f;`. Pour une chaîne, utiliser `scanf("%s", e.nom);` (sans `&`, un tableau est déjà une adresse) ou `strcpy` de `string.h` pour une affectation hors saisie.
5. Exploiter le tableau d'enregistrements avec une boucle `for` : `classe[i].moyenne`.

### EXERCICE RÉSOLU 11 Gestion des élèves d'un lycée de Douala

Énoncé (type Bac). Un lycée souhaite informatiser le suivi de ses élèves de Terminale TI. Écrire un programme C qui : déclare un enregistrement `Eleve` (matricule, nom, moyenne); saisit les informations de  $N$  élèves ( $N \leq 100$ ); affiche le nom et la moyenne du major de la classe.

**SOLUTION**

Solution détaillée.

Étape 1 — Modélisation. L'enregistrement regroupe trois champs de types différents ; le major est l'élève dont la moyenne est maximale : on parcourt le tableau en mémorisant l'indice du maximum.

Étape 2 — Programme.

```
#include <stdio.h>

struct Eleve {
    int matricule;
    char nom[30];
    float moyenne;
};

int main(void)
{
    struct Eleve classe[100];
    int n, i, indiceMajor = 0;

    printf("Nombre d'eleves (<= 100) : ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Eleve %d - matricule : ", i + 1);
        scanf("%d", &classe[i].matricule);
        printf("Nom : ");
        scanf("%s", classe[i].nom); // pas de & : tableau de char
        printf("Moyenne : ");
        scanf("%f", &classe[i].moyenne);
    }

    for (i = 1; i < n; i++) { // recherche du maximum
        if (classe[i].moyenne > classe[indiceMajor].moyenne) {
            indiceMajor = i;
        }
    }

    printf("Major : %s avec %.2f / 20\\",
        classe[indiceMajor].nom, classe[indiceMajor].moyenne);
    return 0;
}
```

Étape 3 — Trace avec 3 élèves : (101, « Ngo », 12,5), (102, « Kamga », 16,0), (103, « Etoundi », 14,75).

$i = 1 : 16,0 > 12,5 \Rightarrow \text{indiceMajor} = 1$

$i = 2 : 14,75 < 16,0 \Rightarrow \text{indiceMajor} \text{ inchangé}$

Affichage : Major : Kamga avec 16.00 / 20.

Étape 4 — Justifications. On initialise `indiceMajor` à 0 et on démarre la boucle à  $i = 1$  pour comparer chaque élément au maximum courant ; la comparaison stricte `>` conserve le premier major en cas d'égalité ; l'accès aux champs se fait via `classe[i].moyenne` car

`classe[i]` est une structure.

Conclusion. Le programme saisit correctement les  $N$  enregistrements et affiche le major : Kamga avec 16,00/20.

## 13 ALGORITHMES CLASSIQUES : RECHERCHE SÉQUENTIELLE ET TRI PAR SÉLECTION

### 13.1 Méthode 5 : Recherche séquentielle et tri par sélection sur un tableau

#### MÉTHODE Les deux algorithmes classiques à maîtriser

Recherche séquentielle (tableau quelconque, trié ou non) :

1. Parcourir le tableau de l'indice 0 à  $n - 1$ .
2. Comparer chaque élément à la valeur cherchée ; dès qu'on la trouve, retourner l'indice et arrêter le parcours.
3. Si la boucle se termine sans trouver, retourner  $-1$  (convention « absent »).

Coût : au plus  $n$  comparaisons.

Tri par sélection (ordre croissant) :

1. Pour  $i$  de 0 à  $n - 2$  : chercher l'indice  $i_{\min}$  du minimum dans la sous-partie non triée  $[i, n - 1]$ .
2. Échanger  $t[i]$  et  $t[i_{\min}]$  (échange en trois temps avec une variable temporaire).
3. La partie  $[0, i]$  est triée définitivement après chaque passage.

Coût :  $\frac{n(n-1)}{2}$  comparaisons, au plus  $n - 1$  échanges. Réflexe : l'échange s'écrit  $tmp = t[i]$ ;  $t[i] = t[i_{\min}]$ ;  $t[i_{\min}] = tmp$ ; — jamais deux affectations sans variable temporaire.

#### EXERCICE RÉSOLU 12 Notes du concours d'entrée à l'ENSP

Énoncé (type Bac). Un tableau  $t$  contient les  $n$  notes ( $n \leq 200$ ) des candidats à un concours. 1) Écrire une fonction `int recherche(float t[], int n, float note)` qui retourne l'indice de la première occurrence de `note`, ou  $-1$  si elle est absente. 2) Écrire une fonction `void triSelection(float t[], int n)` qui trie le tableau par ordre croissant avec le tri par sélection. 3) Donner le nombre de comparaisons effectuées par le tri pour  $n = 50$ .

**SOLUTION**

Solution détaillée.

1) Recherche séquentielle.

```
int recherche(float t[], int n, float note)
{
    int i;
    for (i = 0; i < n; i++){
        if (t[i] == note){
            return i; // trouve : on sort immédiatement
        }
    }
    return -1; // absent
}
```

Justification : le parcours est exhaustif (de 0 à  $n-1$ ) et le `return` garantit l'arrêt à la première occurrence ; si aucune comparaison n'est vraie, on atteint la fin et on retourne -1.

2) Tri par sélection.

```
void triSelection(float t[], int n)
{
    int i, j, imin;
    float tmp;
    for (i = 0; i < n - 1; i++){
        imin = i; // minimum suppose
        for (j = i + 1; j < n; j++){ // recherche du vrai minimum
            if (t[j] < t[imin]) {
                imin = j;
            }
        }
        if (imin != i) { // echange en trois temps
            tmp = t[i];
            t[i] = t[imin];
            t[imin] = tmp;
        }
    }
}
```

Trace sur  $t = [14, 9, 12]$  : passage  $i = 0$  : minimum de  $[14, 9, 12] = 9$  (indice 1) → échange →  $[9, 14, 12]$  ; passage  $i = 1$  : minimum de  $[14, 12] = 12$  (indice 2) → échange →  $[9, 12, 14]$ . Le tableau est trié.

3) Nombre de comparaisons pour  $n = 50$ . La boucle interne effectue  $n-1-i$  comparaisons au passage  $i$  ; le total est :

$$\sum_{i=0}^{n-2} (n-1-i) = (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$$

Pour  $n = 50$  :  $\frac{50 \times 49}{2} = 1225$  comparaisons (et au plus 49 échanges).

Conclusion. La fonction `recherche` retourne l'indice de la première occurrence ou -1, le `triSelection` ordonne le tableau en  $\frac{n(n-1)}{2}$  comparaisons, soit 1225 pour 50 candidats.

## 14 RÉCURSIVITÉ, GESTION DES ERREURS ET PROJET D'APPLICATION COMPLÈTE

### 14.1 Méthode 6 : Écrire une fonction récursive et produire une application complète

#### MÉTHODE Concevoir une fonction récursive sans erreur

1. Trouver le cas de base (condition d'arrêt) : la valeur pour laquelle le résultat est connu directement. Exemple :  $0! = 1$ .
2. Trouver la relation de récurrence : exprimer le problème en fonction d'un sous-problème plus petit. Exemple :  $n! = n \times (n - 1)!$ .
3. Rédiger la fonction : tester le cas de base en premier avec un `if`, puis le cas général avec un appel récursif.
4. Vérifier la convergence : chaque appel doit se rapprocher du cas de base (paramètre qui décroît strictement), sinon pile d'appels infinie et plantage (stack overflow).
5. Tracer l'exécution sur une petite valeur pour valider : empiler les appels, puis dépiler les résultats.

Projet d'application complète (calculatrice) : menu avec `do...while` + `switch`, une fonction par opération, saisie contrôlée, gestion de la division par zéro, compilation et tests systématiques de chaque option du menu.

#### EXERCICE RÉSOLU 13 Factorielle récursive et calculatrice complète

Énoncé (type Bac). 1) Écrire une fonction récursive `long factorielle(int n)` qui calcule  $n!$  pour  $n \geq 0$ . 2) Donner la trace de `factorielle(4)`. 3) Que se passe-t-il si l'utilisateur appelle `factorielle(-3)` avec votre code? Proposer une correction. 4) Intégrer cette fonction dans une calculatrice complète à menu (addition, soustraction, multiplication, division, factorielle, quitter).

#### SOLUTION

Solution détaillée.

1) Fonction récursive. Cas de base :  $0! = 1$ . Récurrence :  $n! = n \times (n - 1)!$ .

```
long factorielle(int n)
{
    if (n <= 1) {           // cas de base : 0! = 1! = 1
        return 1;
    }
}
```

```
    return n * factorielle(n - 1); // cas general
}
```

2) Trace de *factorielle(4)*. Empilement puis dépilement :

$$\begin{aligned}
 \text{factorielle}(4) &= 4 \times \text{factorielle}(3) \\
 &= 4 \times (3 \times \text{factorielle}(2)) \\
 &= 4 \times (3 \times (2 \times \text{factorielle}(1))) \\
 &= 4 \times (3 \times (2 \times 1)) = 24
 \end{aligned}$$

Le résultat retourné est bien  $24 = 4!$ .

3) Appel *factorielle(-3)*. La condition  $n \leq 1$  est vraie dès le premier appel ( $-3 \leq 1$ ), donc la fonction retourne 1 : pas de plantage, mais un résultat mathématiquement faux (la factorielle n'est pas définie pour  $n < 0$ ). Correction : valider la saisie dans le programme principal (refuser tout entier négatif) ou retourner une valeur d'erreur :

```
long factorielle(int n)
{
    if (n < 0) return -1; // code d'erreur
    if (n <= 1) return 1;
    return n * factorielle(n - 1);
}
```

4) Calculatrice complète.

```
#include <stdio.h>

long factorielle(int n)
{
    if (n < 0) return -1;
    if (n <= 1) return 1;
    return n * factorielle(n - 1);
}

int main(void)
{
    int choix, n;
    float a, b;

    do{
        printf("\n --- CALCULATRICE ---\n");
        printf("1.Addition 2.Soustraction 3.Multiplcation\n");
        printf("4.Division 5.Factorielle 0.Quitter\ Choix : ");
        scanf("%d", &choix);

        switch(choix){
            case 1:
                printf("a b : "); scanf("%f %f", &a, &b);
                printf("= %.2f\n", a + b); break;
            case 2:
                printf("a b : "); scanf("%f %f", &a, &b);
                printf("= %.2f\n", a - b); break;
            case 3:
                printf("a b : "); scanf("%f %f", &a, &b);
```

```

        printf("= %.2f\\ ", a * b); break;
    case 4:
        printf("a b : "); scanf("%f %f", &a, &b);
        if (b != 0) printf("= %.2f\\ ", a / b);
        else printf("Erreur : division par zero\\ ");
        break;
    case 5:
        printf("n : "); scanf("%d", &n);
        if (factorielle(n) != -1)
            printf("%d! = %ld\\ ", n, factorielle(n));
        else printf("Erreur : n negatif\\ ");
        break;
    case 0:
        printf("Au revoir !\\ "); break;
    default:
        printf("Choix invalide.\\ ");
    }
} while (choix != 0);

return 0;
}

```

Justifications : le `do...while` garantit l'affichage du menu au moins une fois et la répétition jusqu'au choix 0 ; chaque `case` se termine par `break` pour éviter l'exécution en cascade ; la division est protégée par le test `b != 0` ; le cas d'erreur de la factorielle est intercepté.

Conclusion. La fonction récursive est correcte ( $4! = 24$ ), sécurisée contre les entrées négatives, et la calculatrice complète compile et fonctionne : le projet d'application mono-poste est abouti.

#### ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. Oublier le `&` dans `scanf`. Écrire `scanf("%d", age);` au lieu de `scanf("%d", &age);` : le programme compile parfois mais plante ou produit des valeurs aberrantes à l'exécution. Exception à connaître : les chaînes de caractères (`scanf("%s", nom);`) ne prennent pas de `&`.
2. Confondre `=` (affectation) et `==` (comparaison). `if (note = 10)` affecte 10 à `note` et la condition est toujours vraie : bug logique majeur, très pénalisé dans une copie.
3. Oublier le `break` dans un `switch`. Sans `break`, l'exécution « tombe » dans les cas suivants (fall-through) : plusieurs blocs s'exécutent au lieu d'un seul. Chaque `case` doit se terminer par `break`, sauf volonté explicite justifiée.
4. Division entière involontaire. `moyenne = somme / effectif;` avec deux `int` tronque le résultat ( $34/3$  donne 11 et non 11,33). Il faut qu'au moins un opérande soit réel : déclarer `somme` en `float` ou écrire `(float) somme / effectif` (conversion explicite, `cast`).
5. Fonction récursive sans cas de base (ou qui ne converge pas). Oublier `if (n <= 1) return 1;` ou appeler `factorielle(n)` au lieu de `factorielle(n - 1)` provoque une ré-

ursion infinie et un débordement de pile (stack overflow). Toujours écrire le cas de base en premier et vérifier que le paramètre décroît vers lui. Erreur associée : boucle `while` dont la condition ne change jamais (boucle infinie) — la variable de contrôle doit être modifiée dans le corps de la boucle.

## 15 EXERCICES

### 15.1 Je vérifie mes connaissances

#### EXERCICE 7 QCM : à chaud sur le langage C

Pour chaque question, une seule réponse est correcte. Entoure la bonne lettre.

1. La fonction `printf` est déclarée dans la bibliothèque :  
a) `math.h`   b) `stdio.h`   c) `stdlib.h`   d) `string.h`
2. Pour saisir un entier dans la variable `age`, on écrit :  
a) `scanf("%d", age);`   b) `scanf("%d", &age);`   c) `printf("%d", &age);`   d) `scanf(age);`
3. Après l'exécution de `int x = 7 / 2;`, la variable `x` contient :  
a) 3,5   b) 3   c) 4   d) une erreur de compilation
4. L'expression `(x >= 0) && (x <= 20)` est vraie lorsque :  
a) `x` est strictement négatif   b) `x` appartient à l'intervalle `[0; 20]`   c) `x` vaut exactement 0 ou 20   d) `x` est quelconque
5. Une fonction récursive est une fonction qui :  
a) ne retourne aucune valeur   b) est appelée dans `main`   c) s'appelle elle-même   d) possède plusieurs paramètres

#### EXERCICE 8 Vrai ou faux, avec justification

Réponds par VRAI ou FAUX, puis justifie ta réponse en une ou deux phrases.

1. En C, toute variable doit être déclarée avant d'être utilisée.
2. Les instructions `i++` et `++i` ont exactement le même effet dans toutes les situations.
3. Dans un `switch`, l'oubli du `break` provoque une erreur de compilation.
4. Une variable déclarée à l'intérieur d'une fonction est visible dans toutes les autres fonctions du programme.
5. La boucle `do...while` exécute son corps au moins une fois, même si la condition est fausse dès le départ.
6. L'opérateur `%` (modulo) peut s'appliquer à des opérandes de type `float`.

**EXERCICE 9 Définitions et vocabulaire**

1. Définis les termes suivants : **compilation**, **variable locale**, **variable globale**, **enregistrement** (struct).
2. Quelle est la différence entre la déclaration (prototype) d'une fonction et sa définition?
3. Cite les trois étapes de la démarche du tri par sélection appliqué à un tableau.
4. Qu'appelle-t-on condition d'arrêt dans une fonction récursive, et que se passe-t-il si elle est absente?

**EXERCICE 10 Calculs directs : évaluation d'expressions**

On donne les déclarations suivantes :

```
int a = 10, b = 3, i = 5;  
float x = 10, y = 3;
```

Donne la valeur de chacune des expressions ci-dessous, en précisant son type (int ou float) :

1. `a / b` ; `x / y` ; `a % b`
2. `(float)a / b`
3. `a += b` (donne aussi la nouvelle valeur de `a`)
4. `i++ + 2` puis `++i + 2` (donne la valeur de l'expression et celle de `i` après chaque évaluation)
5. `(a > b) && !(b == 0)`
6. `sqrt(16) + abs(-7)` (on suppose `math.h` inclus)

## 15.2 Je m'entraîne

**EXERCICE 11 Lecture de code : prédire l'affichage**

On donne le programme suivant :

```
#include <stdio.h>  
  
int main(void)  
{  
    int i = 1, s = 0;  
    s += i++;      /* ligne A */  
}
```

```
s += ++i;      /* ligne B */
printf("i = %d, s = %d\n", i, s);

int note = 2;
switch (note) {
    case 1: printf("Passable\n");
    case 2: printf("Bien\n");
    case 3: printf("Tres bien\n");
    break;
    default: printf("Non classe\n");
}
return 0;
}
```

1. Sans exécuter le programme, prédis exactement ce qui s'affiche à l'écran (ligne par ligne). Justifie l'effet des lignes A et B.
2. Explique pourquoi le `switch` affiche plusieurs lignes alors que `note` vaut 2.
3. Que faudrait-il modifier pour n'afficher que « Bien » ?
4. Vérifie tes prédictions en exécutant le programme pas à pas dans Code ::Blocks ou Dev-C++.

#### EXERCICE 12 Équation du second degré

Écris un programme C complet qui :

1. saisit trois réels  $a$ ,  $b$  et  $c$  (coefficients de l'équation  $ax^2 + bx + c = 0$ , avec  $a \neq 0$  supposé);
2. calcule le discriminant  $\Delta = b^2 - 4ac$ ;
3. affiche, à l'aide de `if` imbriqués et de la fonction `sqrt` de `math.h` :
  - les deux solutions réelles distinctes si  $\Delta > 0$ ;
  - la solution double si  $\Delta = 0$ ;
  - le message « Pas de solution réelle » si  $\Delta < 0$ .
4. Teste ton programme avec  $(a, b, c) = (1, -3, 2)$ , puis  $(1, 2, 1)$ , puis  $(1, 0, 1)$ .

#### EXERCICE 13 Table de multiplication et somme des entiers

Écris un programme C complet qui :

1. saisit un entier  $N$  strictement positif (redemander la saisie avec une boucle `do...while` tant que  $N \leq 0$ );
2. affiche la table de multiplication de  $N$  de 1 à 10 sous la forme  $N \times k = \dots$  à l'aide d'une

boucle `for`;

3. calcule et affiche la somme  $S = 1 + 2 + \dots + N$  à l'aide d'une boucle `while`;
4. vérifie le résultat en affichant aussi la valeur de la formule  $\frac{N(N+1)}{2}$  et en comparant les deux valeurs.

#### EXERCICE 14 Correction guidée : chasse aux erreurs

Le programme suivant, censé calculer la moyenne de trois notes, contient six erreurs de compilation classiques. Retrouve-les, corrige-les et justifie chaque correction en une phrase.

```
#include <stdio.h>

int main()
{
    float n1, n2, n3, moyenne;
    printf("Entrez trois notes : ");
    scanf("%f %f %f", n1, n2, n3);
    moyenne = (n1 + n2 + n3) / 3;
    if (moyenne = 10) {
        printf("Admis\n");
    }
    printf("Moyenne = %d\n", moyenne);
    return 0
}
```

Indication : pense aux points-virgules, aux parenthèses de `main`, aux arguments de `scanf`, à la comparaison et au format d'affichage.

### 15.3 Je me prépare au Bac

#### EXERCICE 15 Bac blanc : gestion des notes d'une classe de Terminale TI

Le proviseur d'un lycée de Yaoundé souhaite automatiser le traitement des notes de la classe de Terminale TI. On te demande d'écrire un programme C qui gère les notes sur 20 de 10 élèves.

1. Déclare un tableau `notes` de 10 réels et écris la saisie des 10 notes avec une boucle `for`. Chaque note doit être contrôlée : tant qu'elle n'est pas comprise entre 0 et 20, on la redemande (boucle `do...while`).
2. Écris une fonction `float moyenne(float t[], int n)` qui retourne la moyenne des  $n$  notes du tableau, et appelle-la dans `main`.
3. Écris une fonction `int recherche(float t[], int n, float val)` qui effectue une re-

**cherche séquentielle** de la note `val` et retourne l'indice de sa première occurrence, ou -1 si elle est absente. Teste-la avec une note saisie au clavier.

4. Écris une procédure `void triSelection(float t[], int n)` qui trie les notes par ordre croissant selon l'algorithme du **tri par sélection**. Rappelle le principe de ce tri en trois phrases avant de le coder.
5. Après le tri, affiche le classement : pour chaque note, son rang (1<sup>er</sup>, 2<sup>e</sup>, ...).
6. Application numérique : on donne les notes 12 ; 8 ; 15 ; 8 ; 19 ; 10 ; 14 ; 6 ; 11 ; 16. Déroule à la main les deux premières passes du tri par sélection (montre l'état du tableau après chaque échange), puis donne la moyenne de la classe et le rang de la note 14.

#### EXERCICE 16 Bac blanc : facturation dans une boutique de Douala

Une boutique de vente de téléphones à Douala veut informatiser sa facturation. Pour chaque client, on enregistre : son nom, la quantité d'articles achetés et le prix unitaire en FCFA. Une remise de 5 % est accordée si le montant brut dépasse 100000 FCFA.

1. Définis en C une structure `struct Client` contenant les champs : `nom` (chaîne de 30 caractères), `quantite` (entier) et `prixUnitaire` (réel).
2. Écris une fonction `float montantNet(struct Client c)` qui calcule et retourne le montant net à payer (montant brut diminué de la remise éventuelle). Précise si le paramètre est transmis par valeur ou par adresse, et ce que cela implique.
3. Dans le `main`, saisis les informations de 5 clients rangés dans un tableau de structures, puis affiche pour chacun son nom et son montant net.
4. Affiche enfin le nom du client qui a le montant net le plus élevé, ainsi que le chiffre d'affaires total de la boutique.
5. La patronne souhaite offrir un bonus : le montant net est calculé par une fonction récursive `float puissance(float x, int n)` utilisée pour appliquer un coefficient. Écris cette fonction récursive (avec  $x^0 = 1$  et  $x^n = x \times x^{n-1}$ ), puis trace à la main les appels successifs pour `puissance(2, 5)` en indiquant chaque appel et chaque valeur retournée.

#### EXERCICE 17 Bac blanc : calculatrice à menu pour un cybercafé

Le gérant d'un cybercafé de Bafoussam veut une petite calculatrice en C pour ses opérations quotidiennes. Le programme affiche le menu suivant et se répète tant que l'utilisateur ne choisit pas de quitter :

| Choix | Opération      |
|-------|----------------|
| 1     | Addition       |
| 2     | Soustraction   |
| 3     | Multiplication |
| 4     | Division       |
| 0     | Quitter        |

1. Écris quatre fonctions `addition`, `soustraction`, `multiplication` et `division`, chacune prenant deux réels en paramètres et retournant un réel. Pour la division, explique pourquoi le test du diviseur nul ne peut pas être fait proprement à l'intérieur de la fonction si celle-ci doit retourner un réel, et propose une solution (test dans le `main` ou paramètre supplémentaire).
2. Écris le `main` : une boucle `do...while` affiche le menu, lit le choix, et un `switch` appelle la fonction correspondante après saisie des deux opérandes. Le choix 0 termine le programme ; tout autre choix invalide affiche un message d'erreur et réaffiche le menu.
3. Pour la division, affiche « Division par zero impossible » si le diviseur est nul, sans interrompre le programme.
4. Justifie le choix de la boucle `do...while` plutôt que `while` pour ce programme.
5. Le gérant veut compter le nombre d'opérations effectuées pendant la session. Ajoute une variable globale `compteur`, incrémentée à chaque opération réussie, et affiche sa valeur à la sortie du programme. Rappelle la différence entre variable locale et variable globale, et cite un inconvénient de l'usage des variables globales.

## 16 CORRIGÉS DES EXERCICES

### 16.1 Corrigés de la section « Je vérifie mes connaissances »

#### **CORRIGÉ** Exercice 1 — QCM : à chaud sur le langage C

1. Réponse b) : `printf` est déclarée dans `stdio.h` (standard input/output). La bibliothèque `math.h` contient les fonctions mathématiques (`sqrt`, `cos`, ...).
2. Réponse b) : `scanf("%d", &age);`. La fonction `scanf` a besoin de l'adresse de la variable (opérateur `&`) pour pouvoir y écrire la valeur saisie. Oublier le `&` est l'erreur classique qui provoque un plantage à l'exécution.
3. Réponse b) : 3. Les deux opérandes de `7 / 2` sont des entiers : c'est une division entière (quotient euclidien), la partie décimale est tronquée. Pour obtenir 3,5, il faudrait écrire `7.0 / 2` ou `(float)7 / 2`.
4. Réponse b) : l'expression est vraie si et seulement si  $x \geq 0$  et  $x \leq 20$ , c'est-à-dire  $x \in [0; 20]$ .
5. Réponse c) : une fonction récursive est une fonction qui s'appelle elle-même, avec une condition d'arrêt pour garantir la terminaison.

#### **CORRIGÉ** Exercice 2 — Vrai ou faux, avec justification

1. VRAI. Le C est un langage à déclaration obligatoire : le compilateur doit connaître le type de chaque variable avant sa première utilisation, sinon il signale une erreur du type `undeclared identifier`.
2. FAUX. Utilisées seules comme instruction, elles sont équivalentes. Mais dans une expression, `i++` (post-incrémentation) fournit l'ancienne valeur de `i` puis incrémente, tandis que `++i` (pré-incrémentation) incrémente d'abord puis fournit la nouvelle valeur.
3. FAUX. L'oubli du `break` ne provoque aucune erreur de compilation : c'est une erreur logique. L'exécution « tombe » (fall-through) dans le `case` suivant et exécute ses instructions.
4. FAUX. Une variable déclarée à l'intérieur d'une fonction est locale : elle n'existe et n'est visible que dans cette fonction (plus précisément dans son bloc). Seule une variable déclarée en dehors de toute fonction est globale.
5. VRAI. Dans un `do...while`, le test est effectué après le corps de boucle : celui-ci est donc exécuté au moins une fois, quelle que soit la condition initiale.

6. FAUX. L'opérateur % n'accepte que des opérandes entiers (`int`, `long`, ...). Appliqué à un `float`, il provoque une erreur de compilation. Pour les réels, on utilise `fmod` de `math.h`.

**CORRIGÉ** Exercice 3 — Définitions et vocabulaire

- **Compilation** : traduction du code source C (fichier `.c`) en langage machine exécutable par le processeur, réalisée par un compilateur (par exemple `gcc`), avec détection des erreurs de syntaxe.
  - **Variable locale** : variable déclarée à l'intérieur d'une fonction (ou d'un bloc) ; elle n'existe que pendant l'exécution de cette fonction et n'est visible que dans celle-ci.
  - **Variable globale** : variable déclarée en dehors de toute fonction ; elle est visible et utilisable par toutes les fonctions du programme, et existe pendant toute la durée d'exécution.
  - **Enregistrement** (`struct`) : type composé regroupant sous un même nom plusieurs variables (champs) de types éventuellement différents, par exemple un client avec son nom, sa quantité et son prix unitaire.
- La déclaration (prototype) annonce au compilateur l'interface de la fonction : son type de retour, son nom et les types de ses paramètres, terminée par un point-virgule (ex. : `float moyenne(float t[], int n);`). La définition fournit en plus le corps de la fonction, c'est-à-dire les instructions entre accolades qui réalisent le traitement.
- Les trois étapes du tri par sélection, répétées pour chaque position  $i$  de 0 à  $n - 2$  :
  - Rechercher l'indice du plus petit élément dans la partie non triée du tableau (positions  $i$  à  $n - 1$ );
  - Échanger cet élément avec celui situé à la position  $i$ ;
  - La partie triée s'étend alors d'une position (les  $i + 1$  premiers éléments sont définitivement en place).
- La condition d'arrêt (ou cas de base) est le cas pour lequel la fonction retourne directement un résultat sans s'appeler elle-même (par exemple  $n = 0$  pour la puissance). Si elle est absente (ou jamais atteinte), la fonction s'appelle indéfiniment : la pile d'appels déborde (stack overflow) et le programme plante.

**CORRIGÉ** Exercice 4 — Calculs directs : évaluation d'expressions

- $a / b$  : division entière  $10/3 = 3$  (type `int`).  $x / y$  : division réelle  $10/3 \approx 3,333333$

- (type float).  $a \% b$  : reste de  $10 = 3 \times 3 + 1$ , donc 1 (type int).
- (float)a / b : le cast convertit a en 10,0 avant la division; le résultat est 3,333333 (type float).
  - a += b équivaut à a = a + b : l'expression vaut 13 et a contient désormais 13.
  - i++ + 2 : la post-incrémentation fournit l'ancienne valeur 5, l'expression vaut  $5 + 2 = 7$ , puis i passe à 6. Ensuite ++i + 2 : i est d'abord incrémenté à 7, l'expression vaut  $7 + 2 = 9$ . Valeur finale de i : 7.
  - (a > b) && !(b == 0) : avec les valeurs initiales, a > b est vrai ( $10 > 3$ ) et b est non nul, donc !(b == 0) est vrai. L'expression vaut vrai (1).
  - sqrt(16) + abs(-7) = 4 + 7 = 11. Attention aux types : sqrt retourne un double (4,0) et abs un int (7); la somme est de type double et vaut 11,0.

**ATTENTION** Bibliothèques et abs/fabs

En C standard, abs (entier) est déclarée dans stdlib.h et fabs (réel) dans math.h. Certains compilateurs éducatifs acceptent abs via math.h, mais il est rigoureux d'inclure stdlib.h pour abs(int).

## 16.2 Corrigés de la section « Je m'entraîne »

**CORRIGÉ** Exercice 5 — Lecture de code : prédire l'affichage

1. Affichage produit :

```
i = 3, s = 4
Bien
Tres bien
```

Justification des lignes A et B :

- Ligne A : s += i++; — la post-incrémentation fournit l'ancienne valeur 1 : s = 0 + 1 = 1, puis i passe à 2.
- Ligne B : s += ++i; — la pré-incrémentation incrémente d'abord : i passe à 3, puis s = 1 + 3 = 4.

D'où l'affichage i = 3, s = 4.

- Le case 2 ne contient pas de break : après avoir affiché « Bien », l'exécution tombe dans le case 3 et affiche aussi « Tres bien ». C'est le mécanisme de fall-through. Le break placé après le case 3 arrête ensuite la cascade (le default n'est donc pas exé-

cuté).

- Il suffit d'ajouter un `break`; à la fin du `case 2` (et, par bonne pratique, à la fin du `case 1` aussi) :

```
case 2 : printf("Bien\n");
        break;
```

- Vérification pratique : dans Code ::Blocks, place un point d'arrêt sur la ligne A (clic dans la marge), lance le débogueur (F8), avance pas à pas (F7) et observe l'évolution de `i` et `s` dans la fenêtre des variables (Watches). Les valeurs successives  $i = 2, s = 1$  puis  $i = 3, s = 4$  confirment la prédiction.

### CORRIGÉ Exercice 6 — Équation du second degré

```
#include <stdio.h>
#include <math.h> /* pour sqrt */

int main(void)
{
    float a, b, c, delta, x1, x2;

    printf("Entrez a, b et c (a non nul) : ");
    scanf("%f %f %f", &a, &b, &c);

    delta = b * b - 4 * a * c;

    if (delta > 0) {
        x1 = (-b - sqrt(delta)) / (2 * a);
        x2 = (-b + sqrt(delta)) / (2 * a);
        printf("Deux solutions : x1 = %.2f et x2 = %.2f\n", x1, x2);
    } else {
        if (delta == 0) {
            x1 = -b / (2 * a);
            printf("Solution double : x0 = %.2f\n", x1);
        } else {
            printf("Pas de solution réelle\n");
        }
    }
    return 0;
}
```

Tests :

- $(1, -3, 2) : \Delta = 9 - 8 = 1 > 0$ , solutions  $x_1 = 1$  et  $x_2 = 2$ .
- $(1, 2, 1) : \Delta = 4 - 4 = 0$ , solution double  $x_0 = -1$ .
- $(1, 0, 1) : \Delta = -4 < 0$ , affichage « Pas de solution réelle ».

Points de vigilance : penser à inclure `math.h` et à compiler avec l'option `-lm` sous Linux; bien parenthéser le dénominateur  $(2 * a)$ ; ici les `if` imbriqués sont indispensables car

sqrt d'un nombre négatif est indéfini.

### CORRIGÉ Exercice 7 — Table de multiplication et somme des entiers

```
#include <stdio.h>

int main(void)
{
    int N, k, i, S = 0;

    /* 1. Saisie controlée : N strictement positif */
    do {
        printf("Entrez un entier strictement positif : ");
        scanf("%d", &N);
    } while (N <= 0);

    /* 2. Table de multiplication de N */
    for (k = 1; k <= 10; k++) {
        printf("%d x %d = %d\n", N, k, N * k);
    }

    /* 3. Somme 1 + 2 + ... + N avec une boucle while */
    i = 1;
    while (i <= N) {
        S = S + i;
        i++;
    }
    printf("Somme calculée : S = %d\n", S);

    /* 4. Vérification par la formule */
    printf("Formule N(N+1)/2 = %d\n", N * (N + 1) / 2);
    if (S == N * (N + 1) / 2) {
        printf("Les deux valeurs sont égales : resultat correct.\n");
    }
    return 0;
}
```

Exemple avec  $N = 5$  : la boucle `while` donne  $S = 1 + 2 + 3 + 4 + 5 = 15$  et la formule donne  $\frac{5 \times 6}{2} = 15$  : les deux valeurs coïncident. Le `do...while` garantit que la question est posée au moins une fois et reposée tant que  $N \leq 0$ .

### CORRIGÉ Exercice 8 — Correction guidée : chasse aux erreurs

Les six erreurs, dans l'ordre d'apparition :

1. `float n1, n2, n3, moyenne` : point-virgule manquant en fin de déclaration → `float n1, n2, n3, moyenne;`
2. `scanf("%f %f %f", n1, n2, n3);` : `scanf` attend les adresses des variables →

```
scanf("%f %f %f", &n1, &n2, &n3);.
```

3. `if (moyenne = 10)` : c'est une affectation, pas une comparaison; il faut l'opérateur d'égalité → `if (moyenne == 10)`. (Avec `=`, la condition vaut 10, toujours vraie, et la moyenne est écrasée.)
4. `printf("Admis\n")` : point-virgule manquant → `printf("Admis\n");`.
5. `printf("Moyenne = %d\n", moyenne);` : format incorrect, moyenne est un float → `printf("Moyenne = %.2f\n", moyenne);`.
6. `return 0` : point-virgule manquant → `return 0;`.

Programme corrigé :

```
#include <stdio.h>

int main(void)
{
    float n1, n2, n3, moyenne;
    printf("Entrez trois notes :");
    scanf("%f %f %f", &n1, &n2, &n3);
    moyenne = (n1 + n2 + n3) / 3;
    if (moyenne >= 10) {
        printf("Admis\n");
    }
    printf("Moyenne = %.2f\n", moyenne);
    return 0;
}
```

Remarque : on a aussi écrit `int main(void)` et remplacé `== 10` par `>= 10`, plus logique pour l'admission (une moyenne de 12 doit donner « Admis »).

## 16.3 Corrigés de la section « Je me prépare au Bac »

### **CORRIGÉ** Exercice 9 — Bac blanc : gestion des notes d'une classe de Terminale TI

Principe du tri par sélection (question 4) : pour chaque position  $i$  de 0 à  $n-2$ , on recherche le plus petit élément de la partie non triée (positions  $i$  à  $n-1$ ), on l'échange avec l'élément en position  $i$ , et la partie triée s'étend ainsi d'une position à chaque passe.

Programme complet :

```
#include <stdio.h>

float moyenne(float t[], int n)
{
    float somme = 0;
    int i;
    for (i = 0; i < n; i++) {
        somme += t[i];
    }
}
```

```
    return somme / n;
}

int recherche(float t[], int n, float val)
{
    int i;
    for (i = 0; i < n; i++){
        if (t[i] == val) {
            return i; /* premiere occurrence trouvee */
        }
    }
    return -1; /* absente */
}

void triSelection(float t[], int n)
{
    int i, j, iMin;
    float tmp;
    for (i = 0; i < n - 1; i++){
        iMin = i;
        for (j = i + 1; j < n; j++){
            if (t[j] < t[iMin]) {
                iMin = j;
            }
        }
        if (iMin != i) {
            tmp = t[i]; t[i] = t[iMin]; t[iMin] = tmp;
        }
    }
}

int main(void)
{
    float notes[10], val;
    int i, indice;

    /* 1. Saisie controlee des 10 notes */
    for (i = 0; i < 10; i++){
        do{
            printf("Note de l'eleve %d (0 a 20) : ", i + 1);
            scanf("%f", &notes[i]);
        } while (notes[i] < 0 || notes[i] > 20);
    }

    /* 2. Moyenne */
    printf("Moyenne de la classe : %.2f\n", moyenne(notes, 10));

    /* 3. Recherche sequentielle */
    printf("Note a rechercher : ");
    scanf("%f", &val);
    indice = recherche(notes, 10, val);
    if (indice == -1) {
        printf("Note absente du tableau.\n");
    } else {
        printf("Note trouvee a l'indice %d.\n", indice);
    }
}
```

```

/* 4 et 5. Tri puis classement */
triSelection(notes, 10);
printf("Classement (du plus faible au plus fort) :\n");
for (i = 0; i < 10; i++){
    printf("Rang %d : %.2f\n", i + 1, notes[i]);
}
return 0;
}

```

#### ATTENTION Comparaison de flottants

La fonction `recherche` utilise `t[i] == val`. Comme les notes sont des valeurs entières (0 à 20) stockées dans des `float`, l'égalité exacte fonctionne ici. En général, pour des mesures réelles, on teste `fabs(t[i] - val) < EPSILON` avec une tolérance (ex.  $10^{-6}$ ).

Application numérique — tableau initial : [12; 8; 15; 8; 19; 10; 14; 6; 11; 16].

- Passe 1 ( $i = 0$ ) : le minimum de la partie [0..9] est 6 (indice 7). Échange des positions 0 et 7 :

[6; 8; 15; 8; 19; 10; 14; 12; 11; 16]

- Passe 2 ( $i = 1$ ) : le minimum de la partie [1..9] est 8 (indice 1, déjà en place, aucun échange effectif) :

[6; 8; 15; 8; 19; 10; 14; 12; 11; 16]

Moyenne :  $\frac{12 + 8 + 15 + 8 + 19 + 10 + 14 + 6 + 11 + 16}{10} = \frac{119}{10} = 11,9$ .

Après tri complet : [6; 8; 8; 10; 11; 12; 14; 15; 16; 19]. La note 14 occupe le rang 7 (en comptant à partir du rang 1 pour la plus petite note).

Barème indicatif (20 pts) : saisie contrôlée 3; fonction `moyenne` 3; recherche séquentielle 3; principe du tri énoncé 2; `triSelection` correct 4; affichage du classement 2; application numérique (passes, moyenne, rang) 3.

Points de vigilance : le contrôle  $0 \leq \text{note} \leq 20$  doit utiliser un `do...while` à l'intérieur du `for`; la recherche doit retourner -1 en cas d'absence (valeur sentinelle); la boucle externe du tri s'arrête à  $n - 2$ ; l'échange exige une variable temporaire.

#### CORRIGÉ Exercice 10 — Bac blanc : facturation dans une boutique de Douala

1. Déclaration de la structure :

```

struct Client {
    char nom[30];
    int quantite;
    float prixUnitaire;
}

```

```
| };
```

## 2. Fonction de calcul du montant net :

```
float montantNet(struct Client c)
{
    float brut = c.quantite * c.prixUnitaire;
    if (brut > 100000) {
        return brut * 0.95f; /* remise de 5 % */
    }
    return brut;
}
```

Le paramètre `c` est transmis par valeur : la fonction reçoit une copie de la structure. Conséquence : toute modification de `c` à l'intérieur de la fonction serait sans effet sur le client du `main` (ici on ne fait que lire, donc `c` est correct).

## 3. et 4. Programme principal :

```
#include <stdio.h>

int main(void)
{
    struct Client clients[5];
    int i, iMax = 0;
    float net, caTotal = 0, netMax;

    for (i = 0; i < 5; i++) {
        printf("Client %d - nom : ", i + 1);
        scanf("%29s", clients[i].nom); /* %29s evite le debordement */
        printf("Quantite : ");
        scanf("%d", &clients[i].quantite);
        printf("Prix unitaire (FCFA) : ");
        scanf("%f", &clients[i].prixUnitaire);
    }

    for (i = 0; i < 5; i++) {
        net = montantNet(clients[i]);
        printf("%s doit payer %.0f FCFA\n", clients[i].nom, net);
        caTotal += net;
        if (i == 0 || net > netMax) {
            netMax = net;
            iMax = i;
        }
    }
    printf("Meilleur client : %s(%.0f FCFA)\n",
           clients[iMax].nom, netMax);
    printf("Chiffre d'affaires total : %.0f FCFA\n", caTotal);
    return 0;
}
```

## 4. Fonction récursive et trace :

```
float puissance(float x, int n)
{
```

```

    if (n == 0) {
        return 1;          /* condition d'arrêt */
    }
    return x * puissance(x, n - 1);
}

```

Trace de `puissance(2, 5)` — phase de descente puis de remontée :

$\text{puissance}(2,5) = 2 \times \text{puissance}(2,4)$   
 $\text{puissance}(2,4) = 2 \times \text{puissance}(2,3)$   
 $\text{puissance}(2,3) = 2 \times \text{puissance}(2,2)$   
 $\text{puissance}(2,2) = 2 \times \text{puissance}(2,1)$   
 $\text{puissance}(2,1) = 2 \times \text{puissance}(2,0)$   
 $\text{puissance}(2,0) = 1$  (cas de base)

Remontée :  $\text{puissance}(2,1) = 2$ , puis 4, 8, 16, et enfin  $\text{puissance}(2,5) = 32$ . Vérification :  $2^5 = 32$ .

Barème indicatif (20 pts) : structure 3; `montantNet` avec remise conditionnelle 4; transmission par valeur expliquée 2; saisie du tableau de structures 3; maximum et chiffre d'affaires 3; fonction récursive 3; trace complète 2.

Points de vigilance : la remise ne s'applique que si le brut dépasse strictement 100000 FCFA; pour le maximum, initialiser avec le premier élément (et non 0); la condition d'arrêt  $n = 0$  de la récursivité doit être citée explicitement; `scanf ("%29s", ...)` limite la lecture à 29 caractères pour éviter le débordement de tampon et ne lit toujours pas les espaces dans les noms composés.

### **CORRIGÉ** Exercice 11 — Bac blanc : calculatrice à menu pour un cybercafé

1. Les quatre fonctions :

```

float addition(float a, float b)    { return a + b; }
float soustraction(float a, float b) { return a - b; }
float multiplication(float a, float b) { return a * b; }
float division(float a, float b)    { return a / b; }

```

Pour la division, le test du diviseur nul ne peut pas être fait proprement à l'intérieur de la fonction : celle-ci doit retourner un réel dans tous les cas, et aucune valeur réelle ne peut signifier « erreur » de façon fiable (retourner 0 serait ambigu, car  $0/5 = 0$  est un résultat légitime). Solution : tester le diviseur dans le *main* avant l'appel (ou passer un paramètre supplémentaire, par exemple l'adresse d'un entier indicateur d'erreur).

2. et 3. Programme principal :

```
#include <stdio.h>
```

```
int compteur = 0; /* variable globale (question 5) */

float addition(float a, float b)    { return a + b; }
float soustraction(float a, float b) { return a - b; }
float multiplication(float a, float b) { return a * b; }
float division(float a, float b)    { return a / b; }

int main(void)
{
    int choix;
    float a, b;

    do {
        printf("\n--- CALCULATRICE ---\n");
        printf("1. Addition\n2. Soustraction\n");
        printf("3. Multiplication\n4. Division\n0. Quitter\n");
        printf("Votre choix : ");
        scanf("%d", &choix);

        if (choix >= 1 && choix <= 4) {
            printf("Entrez deux nombres : ");
            scanf("%f %f", &a, &b);
        }

        switch (choix) {
            case 1:
                printf("Resultat : %.2f\n", addition(a, b));
                compteur++;
                break;
            case 2:
                printf("Resultat : %.2f\n", soustraction(a, b));
                compteur++;
                break;
            case 3:
                printf("Resultat : %.2f\n", multiplication(a, b));
                compteur++;
                break;
            case 4:
                if (b == 0) {
                    printf("Division par zero impossible\n");
                } else {
                    printf("Resultat : %.2f\n", division(a, b));
                    compteur++;
                }
                break;
            case 0:
                printf("Au revoir !\n");
                break;
            default:
                printf("Choix invalide, recommencez.\n");
        }
    } while (choix != 0);

    printf("Operations effectuees : %d\n", compteur);
    return 0;
}
```

3. Le `do...while` est le bon choix car le menu doit être affiché au moins une fois avant de connaître le choix de l'utilisateur : la condition `choix != 0` ne peut être évaluée qu'après la première saisie. Avec un `while`, il faudrait initialiser artificiellement `choix` à une valeur non nulle avant la boucle.
4. La variable globale `compteur` est déclarée en dehors de toute fonction : elle est visible partout et conserve sa valeur pendant toute la session. Une variable locale, déclarée dans une fonction, n'existe que pendant l'appel de celle-ci et n'est visible que dans son bloc. Inconvénient des globales : elles peuvent être modifiées par n'importe quelle fonction, ce qui rend les erreurs difficiles à tracer et nuit à la lisibilité et à la maintenance du programme ; il vaut mieux, quand c'est possible, passer des paramètres.

Barème indicatif (20 pts) : quatre fonctions 4 ; explication du problème de la division et solution 3 ; boucle `do...while` et menu 3 ; `switch` complet avec `break` et `default` 4 ; gestion du diviseur nul 2 ; justification du `do...while` 2 ; variable globale, affichage final et discussion 2.

Points de vigilance : chaque `case` doit se terminer par `break` ; le compteur ne doit être incrémenté que pour une opération réussie (pas pour une division par zéro ni un choix invalide) ; les prototypes des fonctions doivent figurer avant le `main` si les définitions sont placées après.

## 17 FICHE BILAN

### Programmation en C

#### Structure et types

- `stdio.h` : `printf`, `scanf`
- `int`, `float`, `char`, `double`

#### Opérateurs

- Priorités, casts
- `++`, `+=`, `&&`, `||`

#### Alternatives et boucles

- `if`, `switch`
- `for`, `while`, `do...while`

#### Fonctions et struct

- Portée, paramètres
- Enregistrements

#### Algorithmes classiques

- Recherche séquentielle
- Tri par sélection

#### Récursivité

- Cas de base
- Appels récursifs

#### Compilation et erreurs

- IDE Code ::Blocks
- Exécutable, débogage

Carte mentale de la leçon : les sept compétences clés de la programmation en C.

### FICHE BILAN L'essentiel à connaître pour l'examen

#### Définitions clés

- **Programme C** : suite d'instructions organisée autour de la fonction `main()`, traduite en langage machine par un **compilateur** (ex. GCC via Code ::Blocks).
- **Variable** : zone mémoire nommée et typée (`int`, `float`, `double`, `char`); **constante** : valeur fixe déclarée avec `const` ou `#define`.
- **Enregistrement** (`struct`) : type composé regroupant plusieurs champs de types différents (ex. un élève : nom, âge, moyenne).
- **Fonction** : bloc réutilisable déclaré avec un type de retour, un nom et des paramètres ; **variable locale** (visible dans la fonction) vs **variable globale** (visible partout).
- **Fonction récursive** : fonction qui s'appelle elle-même, avec un **cas de base** qui arrête les appels.

Syntaxe à connaître par cœur

| Notion         | Syntaxe C                                                                               |
|----------------|-----------------------------------------------------------------------------------------|
| Entrée-sortie  | <code>printf("%d", n); scanf("%f", &amp;x);</code>                                      |
| Alternative    | <code>if (cond) {...} else {...}; switch (n) { case 1: ...; break; default: ...}</code> |
| Boucles        | <code>for (i=0; i&lt;n; i++); while (cond); do {...} while (cond);</code>               |
| Fonction       | <code>float moyenne(float a, float b) { return (a+b)/2; }</code>                        |
| Enregistrement | <code>struct Eleve { char nom[30]; float moy; };</code>                                 |
| Maths (math.h) | <code>sqrt(x), abs(n), cos(x), pow(x, y)</code> — compiler avec <code>-lm</code>        |

### Opérateurs et priorités (du plus prioritaire au moins prioritaire)

`() > ++ -- > * / % > + - > < <= > >= > == != > && > || > = += -= *= /=`

Conversion de type : `(float)a / b` force une division réelle; `7 / 2` vaut 3 mais `7.0 / 2` vaut 3,5.

### Méthodes express

1. **Recherche séquentielle** : parcourir le tableau de 0 à  $n - 1$ ; si `t[i] == x`, retourner `i`; sinon retourner `-1`. Coût :  $n$  comparaisons au pire.
2. **Tri par sélection** : pour  $i$  de 0 à  $n - 2$ , chercher l'indice du minimum dans  $[i, n - 1]$ , puis échanger `t[i]` et `t[min]`.
3. **Récursivité** : écrire d'abord le cas de base (ex. `if (n == 0) return 1;`), puis l'appel récursif sur un problème plus petit (`return n * fact(n-1);`).
4. **Corriger une erreur de compilation** : lire le premier message (fichier, ligne), vérifier ; oublié, parenthèses/accolades, variable non déclarée, & dans `scanf`.

### À RETENIR

#### Checklist avant le Bac

- ☐ Je sais écrire la structure minimale d'un programme C (`#include <stdio.h>, int main(), return 0;`).
- ☐ Je maîtrise les formats de `printf/scanf` : `%d, %f, %c, %s`, et le `&` devant la variable lue.
- ☐ Je connais les types de base et je sais déclarer une constante avec `const` ou `#define`.
- ☐ Je distingue division entière et division réelle, et je sais utiliser un cast : `(float)a`.
- ☐ Je cite les opérateurs relationnels et logiques (`==, !=, &&, ||, !`) sans les confondre avec `=`.
- ☐ Je sais choisir entre `if...else`, `switch`, `for`, `while` et `do...while` selon la situation.
- ☐ Je sais déclarer et appeler une fonction, et distinguer variable locale et variable globale.

- ☐ Je sais déclarer un `struct` et accéder à ses champs avec le point : `e.moy`.
- ☐ Je sais écrire la recherche séquentielle et le tri par sélection sur un tableau.
- ☐ Je sais écrire une fonction récursive avec son cas de base (factorielle, somme, Fibonacci).
- ☐ Je sais compiler, exécuter pas à pas dans Code ::Blocks et corriger les erreurs signalées.
- ☐ Je sais produire un programme complet et exécutable (ex. calculatrice, gestion de notes d'une classe).

**Signé • OnBuch+**Ludovic A.  
CONFORME AU PROGRAMME OFFICIEL MINESEC**OnBuch+**